DIGT2107: Practice of Software Development

Project Deliverable 3.3:

Final Project: Full Documentation, Implementation, Presentation

Project Name:

SmartBudget – Personal Finance Tracker

Team Number:

Team #3

Team Members:

Mohsen Pasdar, Muhammed Ahmed, Alia Hagi-Dhaffe

Date:

March 29, 2026

Table of Contents

1 Introduction 5
2 Final Software Requirements Specification (SRS) 6
2.1 Functional Requirements 6
2.2 Non-Functional Requirements 7
3 Final Software Design Document (SDD) 7
3.1 System Design Overview (SDD) 8
3.1.1 Architectural Style and Layering 8
3.1.2 Layer Responsibilities 9
3.1.3 Components and Responsibilities 9
3.1.4 Validation and Error Handling Strategy 10
3.1.5 Design Coverage of Key Use Cases 10
3.2 UML Class Diagrams 10
3.2.1 High-Level Class Diagram (System-Wide View) 10
3.2.2 Class Diagram A — Transactions Module 11
3.2.3 Class Diagram B — Budgets and Alerts Module 12
3.2.4 Class Diagram C — Reports Module 13
3.2.5 Cross-Cutting Classes Used Across Modules 14
3.3 Sequence Diagrams 14
3.3.1 SD-1 — UC-1 Record Transaction (Add Expense/Income) 15
3.3.2 SD-2 — UC-4 Set Monthly Budget + UC-6 Overspending Alert Check 16
3.3.3 SD-3 — UC-5 View Summary Report 17
3.4 Activity Diagrams 18
3.4.1 UC-1 Record Transaction (Add) 19
3.4.2 UC-2 Amend Transaction 20
3.4.3 UC-3 View & Filter Transactions 22
3.4.4 UC-4 Set Monthly Budget 22
3.4.5 UC-4 Edit or Delete Monthly Budget 24
3.4.6 UC-5 View Summary Report 25
3.4.7 Load Saved Data on Startup 26
4 Final Testing Documentation 26
4.1 Testing Objectives and Scope 27

4.2 Testing Methodology and Execution 27
4.3 Test Cases and Traceability 28
4.3.1 TC-001 33
4.3.2 TC-002 34
4.3.3 TC-003 34
4.3.4 TC-004 35
4.3.5 TC-005 35
4.3.6 TC-006 35
4.3.7 TC-007 36
4.3.8 TC-008 36
4.3.9 TC-009 37
4.3.10 TC-010 37
4.3.11 TC-011 38
4.3.12 TC-012 38
4.3.13 TC-013 38
4.3.14 TC-014 39
4.3.15 TC-015 39
4.3.16 TC-016 40
4.3.17 TC-017 40
4.3.18 TC-018 41
4.3.19 TC-019 41
4.3.20 TC-020 41
4.3.21 TC-021 42
4.3.22 TC-022 42
4.3.23 TC-023 43
4.3.24 TC-024 43
4.3.25 TC-025 43
4.3.26 TC-026 44
4.3.27 TC-027 44
4.3.28 TC-028 45
4.3.29 TC-029 45
4.3.30 TC-030 46
4.3.31 TC-031 46

4.3.32 TC-032 46
4.4 Test Results 47
4.4.1 Manual GUI Smoke Test Results 47
4.4.2 Automated Unit Testing Summary 48
4.5 Coverage Summary 48
4.6 Testing Conclusion 49
5 User Manual 49
5.1 Introduction 50
5.2 Getting Started 50
5.2.1 System Requirements 50
5.2.2.2 Clone the Repository 50
5.2.2.3 Maven Wrapper (Recommended) 51
5.2.2.4 Running Inside IntelliJ IDEA 51
5.3 User Interface Guide 51
5.3.1 Transactions Page 51
5.3.2 Budgets Page 52
5.3.3 Reports Page 53
5.3.4 First Launch 53
5.4 Navigating SmartBudget 54
5.5 Transactions 54
5.5.1 Adding a Transaction 54
5.5.2 Editing a Transaction 54
5.5.3 Deleting a Transaction 54
5.5.4 Viewing Transactions 55
5.5.5 Filtering Transactions 55
5.6 Budgets 55
5.6.1 Setting a Budget 55
5.7 Overspending Alerts 55
5.8 Reports 55
5.8.1 Summary Report 55
5.8.2 Category Spending Chart 56
5.9 Data Persistence 56
5.10 Offline Mode and Privacy 56

5.11 Performance & Responsiveness 56
5.12 Cross-Platform Support 56
5.13 Maintenance & Updates 56
5.14 Frequently Asked Questions 56
5.15 Summary 57
6 Code Repository 57
6.1 Repository Link: 57
6.2 Submission Tag 57
7 Conclusion 58

1 Introduction

This document represents Deliverable 3.3, the final project submission for the SmartBudget – Personal Finance Tracker project. SmartBudget is a desktop-based personal finance application designed to help users record daily transactions, manage monthly budgets, receive overspending alerts, and generate summary reports to better understand their spending habits. The main goal of the system is to make personal budgeting simple, practical, and easy to manage through a clear interface and reliable core features.

This final submission brings together the complete work developed across all previous iterations of the project. Earlier deliverables established the project vision, defined the functional and non-functional requirements, documented the use cases and user stories, prepared the testing plan, produced the software design, and implemented the system step by step. In the later stages of development, the project was expanded into a fully functional and integrated application, including persistence support, startup data loading, improved validation, and broader testing of the system's workflows.

The purpose of this final report is to present the completed SmartBudget system together with the full documentation required for the final deliverable. This submission focuses on providing a clear and professional record of the finished project, showing that the system's requirements, design, implementation, testing, and end-user guidance are all fully integrated and aligned with the final version of the software. In this way, the report reflects the complete development journey of the project from early planning to final implementation.

This report is organized into six main chapters. Following this introduction, Chapter 2 presents the final Software Requirements Specification (SRS), which summarizes the finalized functional and non-functional requirements of the system and reflects any updates made during development. Chapter 3 presents the final Software Design Document (SDD), including the architectural overview and the design elements that describe the structure and behavior of the system. Chapter 4 provides the final testing documentation, including the testing approach, test cases, execution results, and evidence of system reliability and correctness. Chapter 5 contains the user manual, which explains how end users can install, navigate, and operate the software. Chapter 6 concludes the report by summarizing the final state of the project and the work completed by the team.

Overall, this final deliverable demonstrates that SmartBudget has been developed into a complete software system supported by comprehensive documentation, final testing, and user

guidance. Together, the chapters in this report present the final version of the project in a form that is organized, consistent, and ready for submission.

2 Final Software Requirements Specification (SRS)

This chapter presents the final Software Requirements Specification (SRS) for SmartBudget – Personal Finance Tracker. The purpose of this chapter is to define the final functional and non-functional requirements of the system in a clear and organized way, and to ensure that these requirements match the completed version of the software. In accordance with the final deliverable requirements, this SRS reflects the implemented features of SmartBudget and incorporates the refinements made during development so that the documented scope remains aligned with the final system.

SmartBudget is a desktop-based personal finance application designed to help users record and manage daily financial activity, plan monthly budgets, receive overspending alerts, and review summary reports. The system is intended to provide a simple and practical way for users to understand their income and expenses through a clear interface and core financial tracking features. The final version of the system preserves the main scope established earlier in the project while bringing the features together into a complete and integrated application.

2.1 Functional Requirements

The final functional requirements of SmartBudget are as follows:

- **FR1 – Transactions:** The system must let the user record, edit, and delete income and expense transactions (amount, date, category, optional note), and persist them locally so they remain available across sessions.
- **FR2 – Browsing & Filters:** The system must let the user view transactions and apply filters (e.g., by date range and category) to review specific spending.
- **FR3 – Budgets & Alerts:** The system must let the user set a monthly spending limit per category and notify the user when spending reaches or exceeds that limit.
- **FR4 – Summaries & Charts:** The system must provide a period summary (income, expenses, balance) and at least one visual breakdown of spending by category.

Together, these functional requirements define the core behavior of SmartBudget and reflect the main features that were carried from the earlier requirements stage into the completed system. The final implementation continues to follow this same feature scope, including transaction management, filtering, budgeting, alerts, reporting, and persistence across sessions.

2.2 Non-Functional Requirements

The final non-functional requirements of SmartBudget are as follows:

- **Usability**: The system should present a simple, consistent JavaFX interface with clear labels, readable fonts, and sensible navigation between Transactions, Budgets, and Reports.
- **Performance**: The system should load typical datasets quickly and remain responsive during common operations.
- **Reliability**: The system should save changes immediately to local storage to minimize data loss and handle invalid inputs gracefully with clear feedback.
- **Privacy/Offline**: The system should operate fully offline and avoid sending data over a network.
- **Portability**: The system should run on Windows, macOS, and Linux with a supported Java runtime.
- **Maintainability**: The system should adopt a modular structure (e.g., MVC) and include unit tests for core business logic to enable straightforward updates.

These non-functional requirements define the expected quality of the SmartBudget system and remain important in the final version because they guide not only what the software does, but also how well it performs, how easy it is to use, and how reliably it behaves during normal operation.

These requirements are based on the earlier requirements analysis and were refined during development to align with the final implemented system.

3 Final Software Design Document (SDD)

This chapter presents the final Software Design Document (SDD) for SmartBudget – Personal Finance Tracker. The purpose of this chapter is to describe the final design of the

system and show how the software is organized to support the required functionality in a clear, maintainable, and testable way. This chapter includes the architectural overview and the main design elements of the system, such as class diagrams, sequence diagrams, and activity diagrams, updated to reflect the final implemented state of SmartBudget. Building on the earlier design work completed during the project, this chapter explains how the system's layers, components, and key workflows fit together and how the design supports the completed features of transaction management, budgeting, overspending alerts, reporting, and persistent data handling.

3.1 System Design Overview (SDD)

3.1.1 Architectural Style and Layering

SmartBudget follows a layered architecture to keep the user interface, business logic, domain entities, and persistence concerns separated. This design improves maintainability and makes the system easier to test and extend. The system is organized into four main layers: **UI (View + Controller), Service (Business Logic), Domain Model (Entities/Results),** and **Persistence (Storage).**

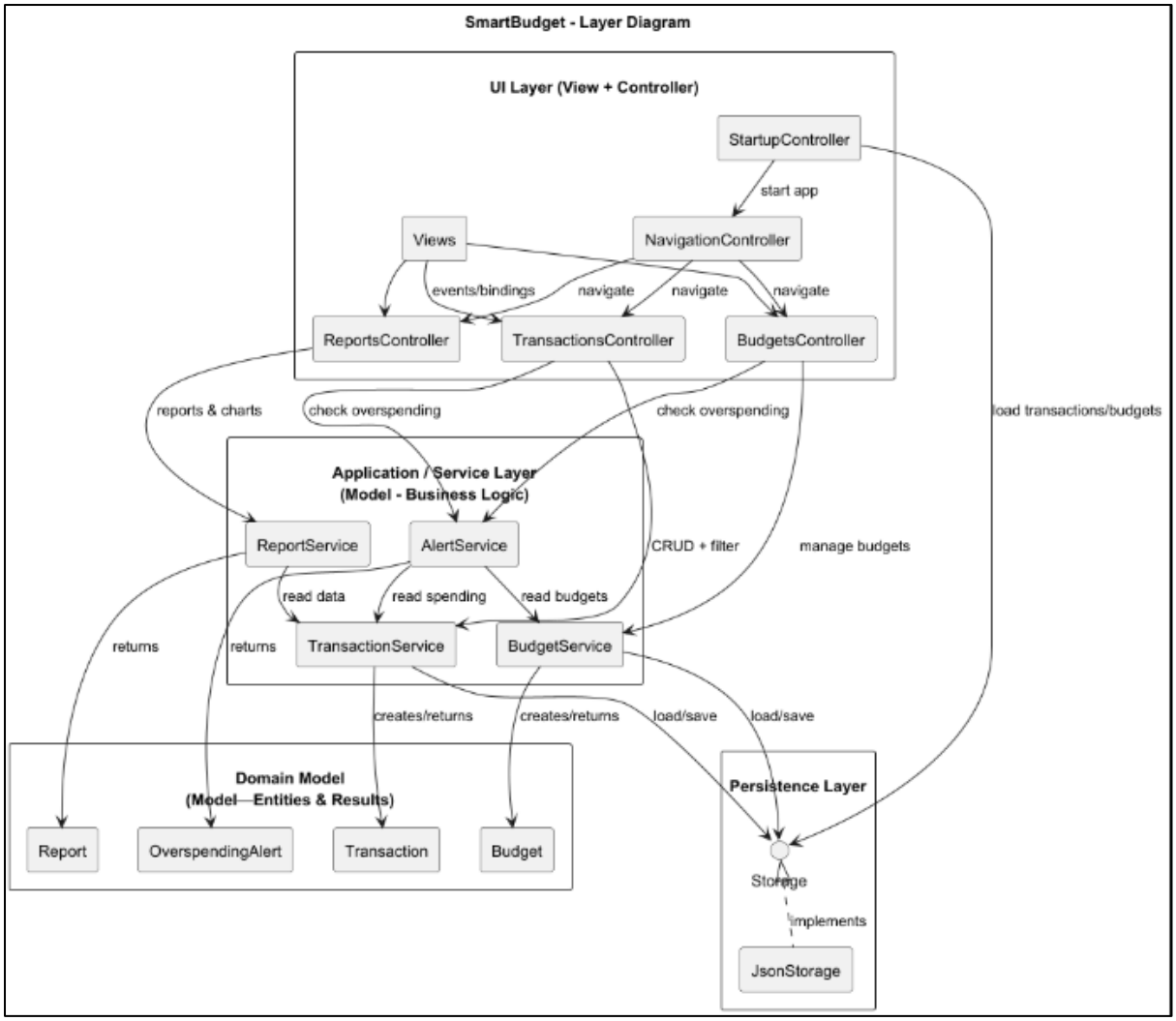


Figure 3-1: Layer diagram

3.1.2 Layer Responsibilities

- **UI Layer (Views + Controllers):** Handles user interactions and navigation. Controllers collect inputs, call services, and update the screen.
- **Service Layer:** Contains the core business rules such as adding transactions, managing budgets, generating reports, and checking overspending alerts.
- **Domain Model:** Represents the system's main data objects (e.g., Transaction, Budget) and report-related result objects (e.g., SummaryReport).
- **Persistence Layer:** Abstracts saving/loading through a **Storage** interface. A JSON-based storage class is designed (JsonStorage), but persistence is not finalized in this iteration.

3.1.3 Components and Responsibilities

The application is centered around three main functional modules:

Transactions module: Records, edits, and filters transactions through TransactionsController and TransactionService.

Budgets + Alerts module: Sets monthly budgets through BudgetsController and BudgetService, and checks overspending through AlertService.

Reports module: Generates summary reports through ReportsController and ReportService.

3.1.4 Validation and Error Handling Strategy

Input validation is handled consistently using a Validator component. Controllers pass user inputs to services, and services call the validator before creating or updating domain objects. If validation fails, the service returns an error response (or throws a validation exception) and the controller displays a clear message to the user.

3.1.5 Design Coverage of Key Use Cases

The design documentation covers the main workflows required by the system, including recording and editing transactions, viewing/filtering transactions, setting/editing budgets, generating summary reports, and loading saved data on startup. The sequence and activity diagrams show both the main paths and important decision points such as invalid input or user cancellation.

3.2 UML Class Diagrams

This section presents the UML class diagrams for SmartBudget. The diagrams describe the system's static structure, including the main classes, their attributes and operations, and how they relate to each other through associations and dependencies. We include a high-level class diagram to give an overall picture of the architecture, followed by three detailed diagrams that break the design into the main modules: Transactions, Budgets & Alerts, and Reports. Together, these diagrams show how responsibilities are separated across controllers, services, domain entities, and storage-related components.

3.2.1 High-Level Class Diagram (System-Wide View)

The high-level class diagram presents the overall structure of SmartBudget, showing how the main controllers interact with their services, how services use domain entities, and how persistence is abstracted through a storage component. This view highlights the separation between UI logic and business logic.

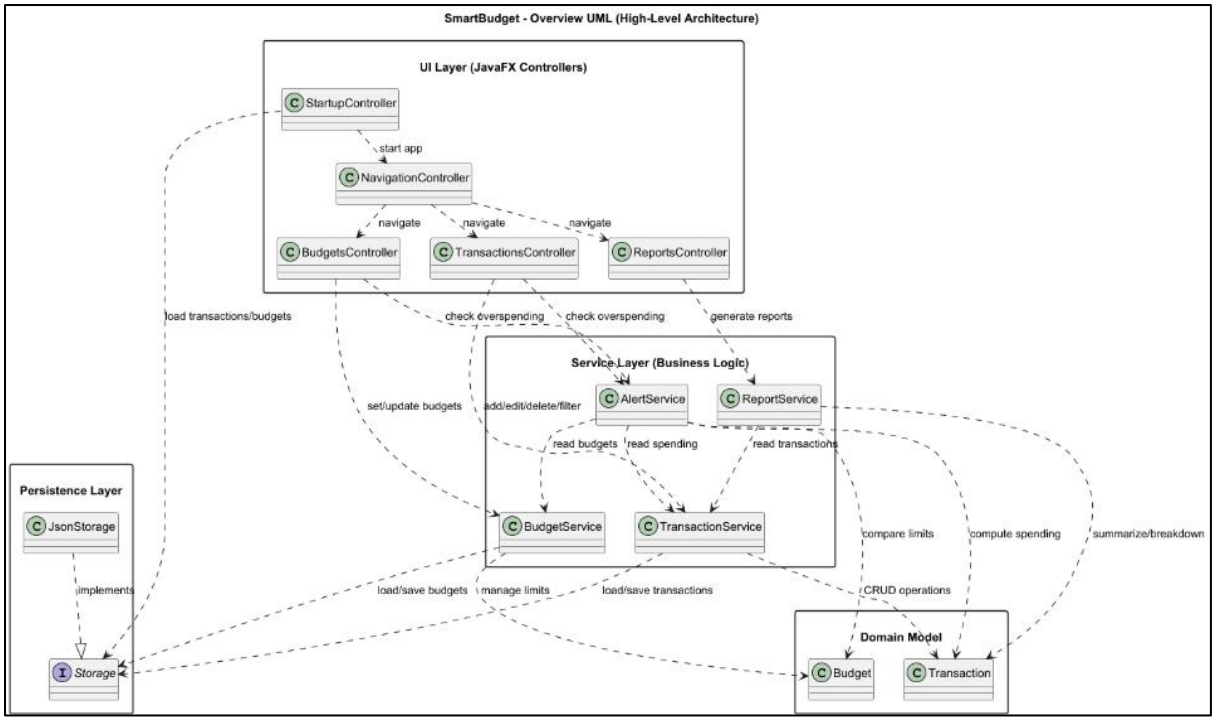


Figure 3-2: High-Level UML Class Diagram

3.2.2 Class Diagram A — Transactions Module

This diagram focuses on transaction-related functionality, including recording and managing transactions. Key classes include *TransactionsController*, *TransactionService*, and the *Transaction* domain entity. It also shows supporting classes such as *TransactionFilter* for filtering logic and shared components like *Validator*, *AlertService*, and *JsonStorage*.

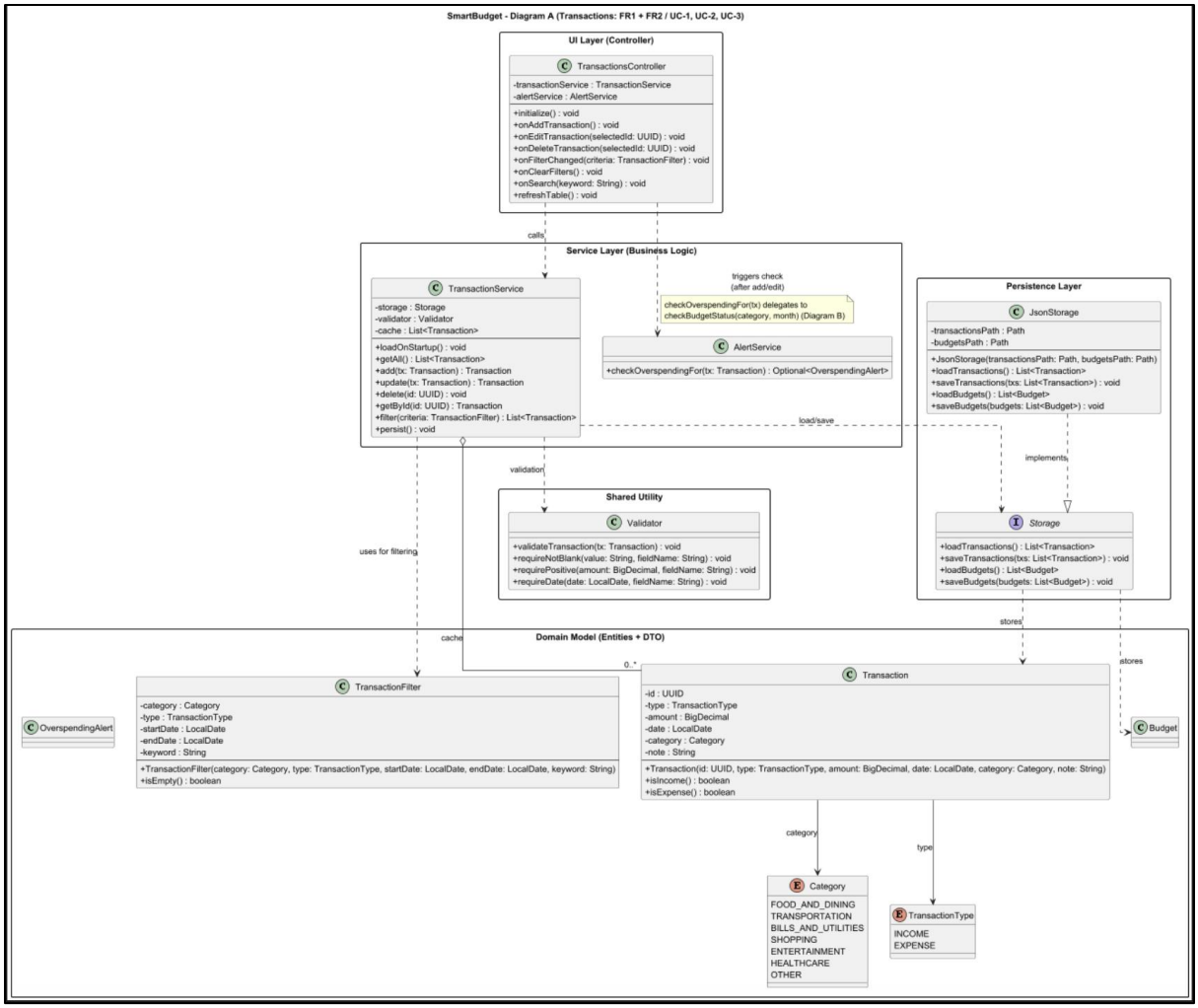


Figure 3-3: UML Class Diagram A — Transactions

3.2.3 Class Diagram B — Budgets and Alerts Module

This diagram focuses on setting budgets and checking overspending alerts. The *BudgetsController* works with *BudgetService* to create or update monthly budgets, while *AlertService* compares budget limits with transaction totals. Shared dependencies include *TransactionService*, *Validator*, and storage (*JsonStorage* / *Storage* interface).

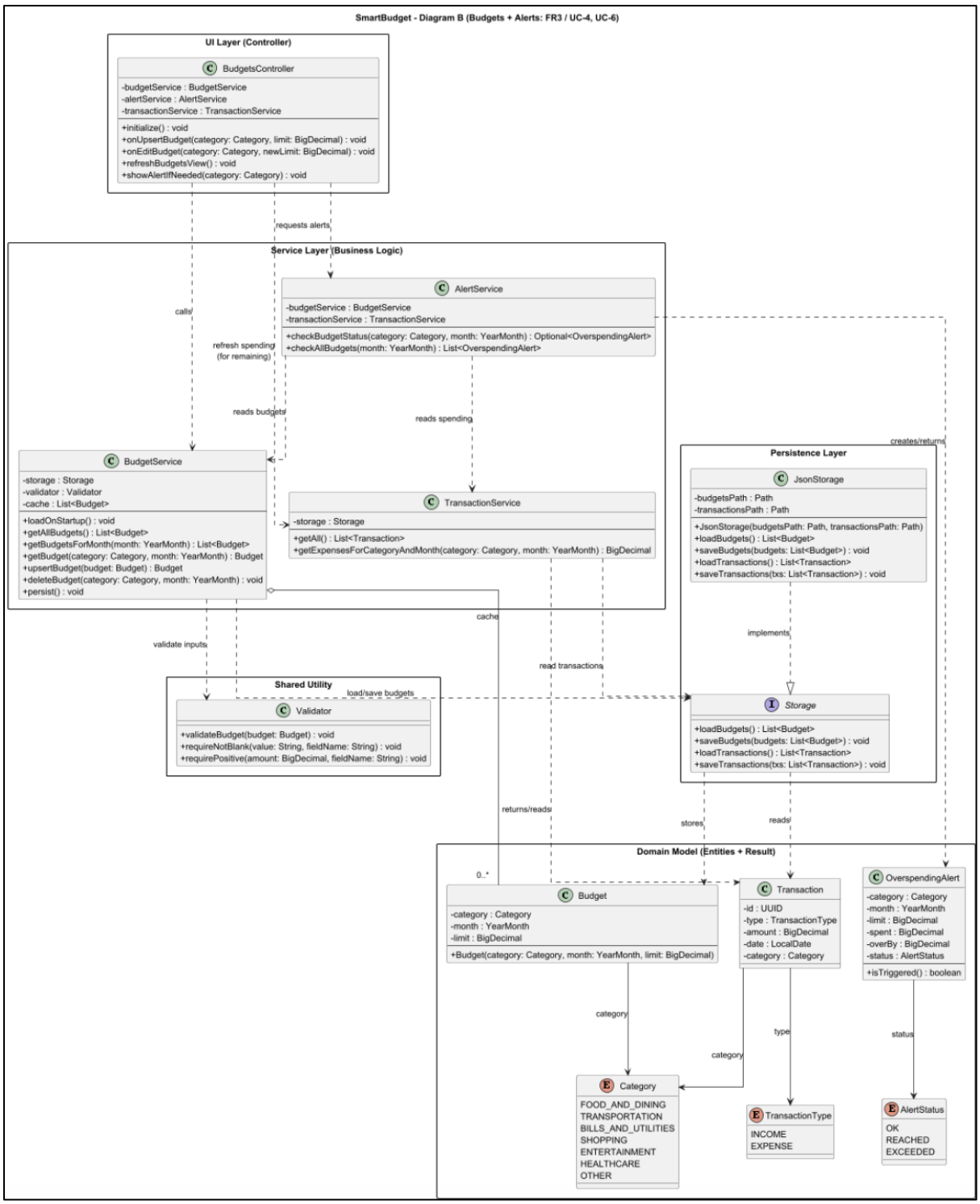


Figure 3-4: UML Class Diagram B — Budgets and Alerts

3.2.4 Class Diagram C — Reports Module

This diagram focuses on reporting features. *ReportsController* requests reports from *ReportService*, which uses *TransactionService* to retrieve transactions in a selected period. Report results are represented through reporting objects such as *SummaryReport*, *CategoryTotal*, and *ReportResult*, which structure the report output in a clean and testable way.

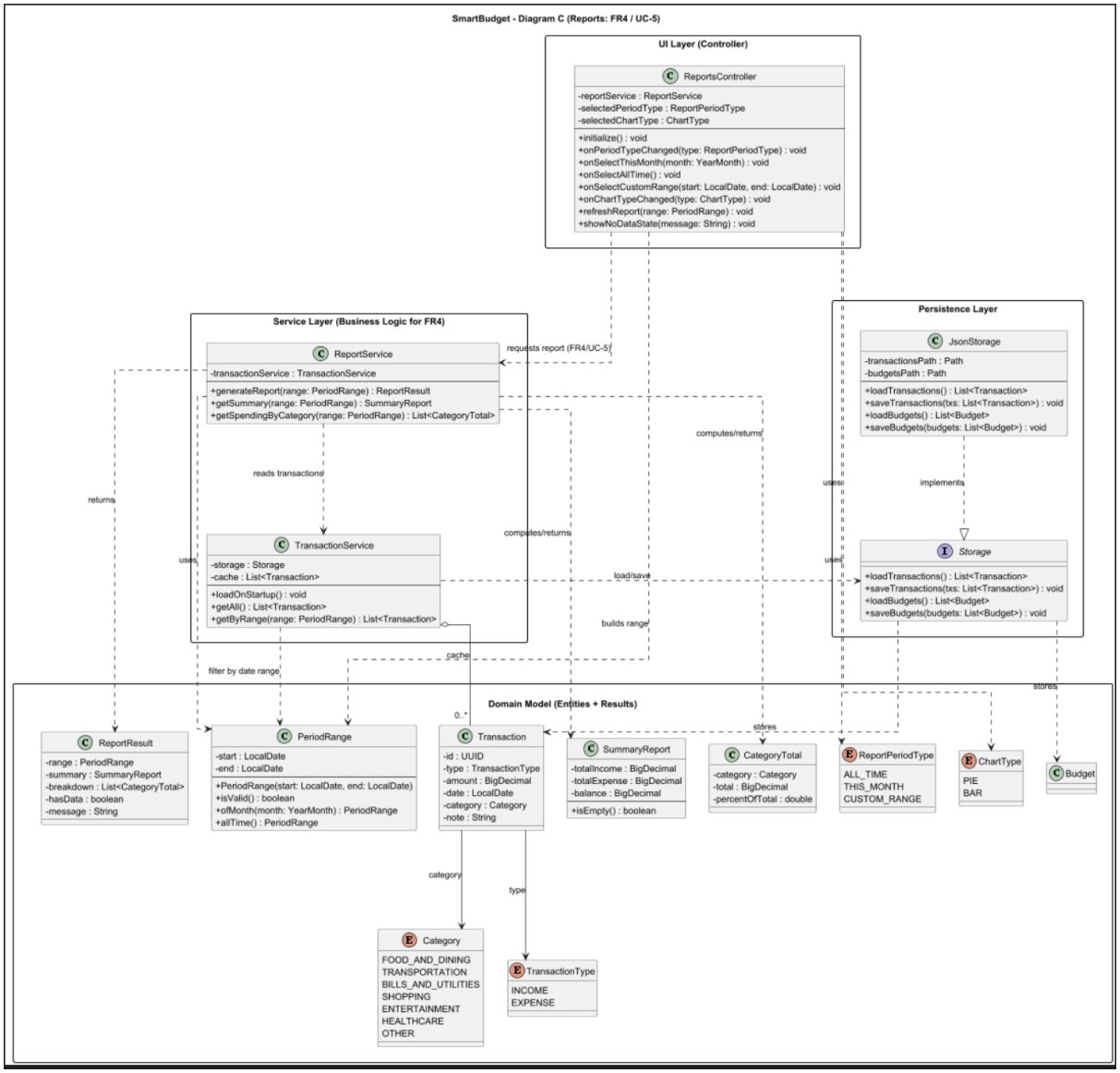


Figure 3-5: UML Class Diagram C — Reports

3.2.5 Cross-Cutting Classes Used Across Modules

Several components are shared across multiple modules:

- **Validator:** Central validation rules used before creating/updating objects.
- **JsonStorage / Storage:** Abstraction for saving and loading data.
- **AlertService:** Used by both transactions and budgets to detect overspending situations.

3.3 Sequence Diagrams

This section includes the sequence diagrams for SmartBudget, which illustrate the system's runtime behavior for key workflows. Each diagram shows the step-by-step interaction

between the user interface (controllers), service layer, and domain objects, including how validation and results are handled. The selected sequence diagrams represent critical use cases and demonstrate how the system processes user requests from the moment the user triggers an action in the GUI to the point where the business logic completes and a response is returned to the UI.

3.3.1 SD-1 — UC-1 Record Transaction (Add Expense/Income)

This sequence diagram shows how a user records a new transaction. The controller collects form inputs, the service validates them, a new *Transaction* is created, and the updated data is returned. The diagram also includes an alternate path for invalid input, where the user receives an error message instead of saving the transaction.

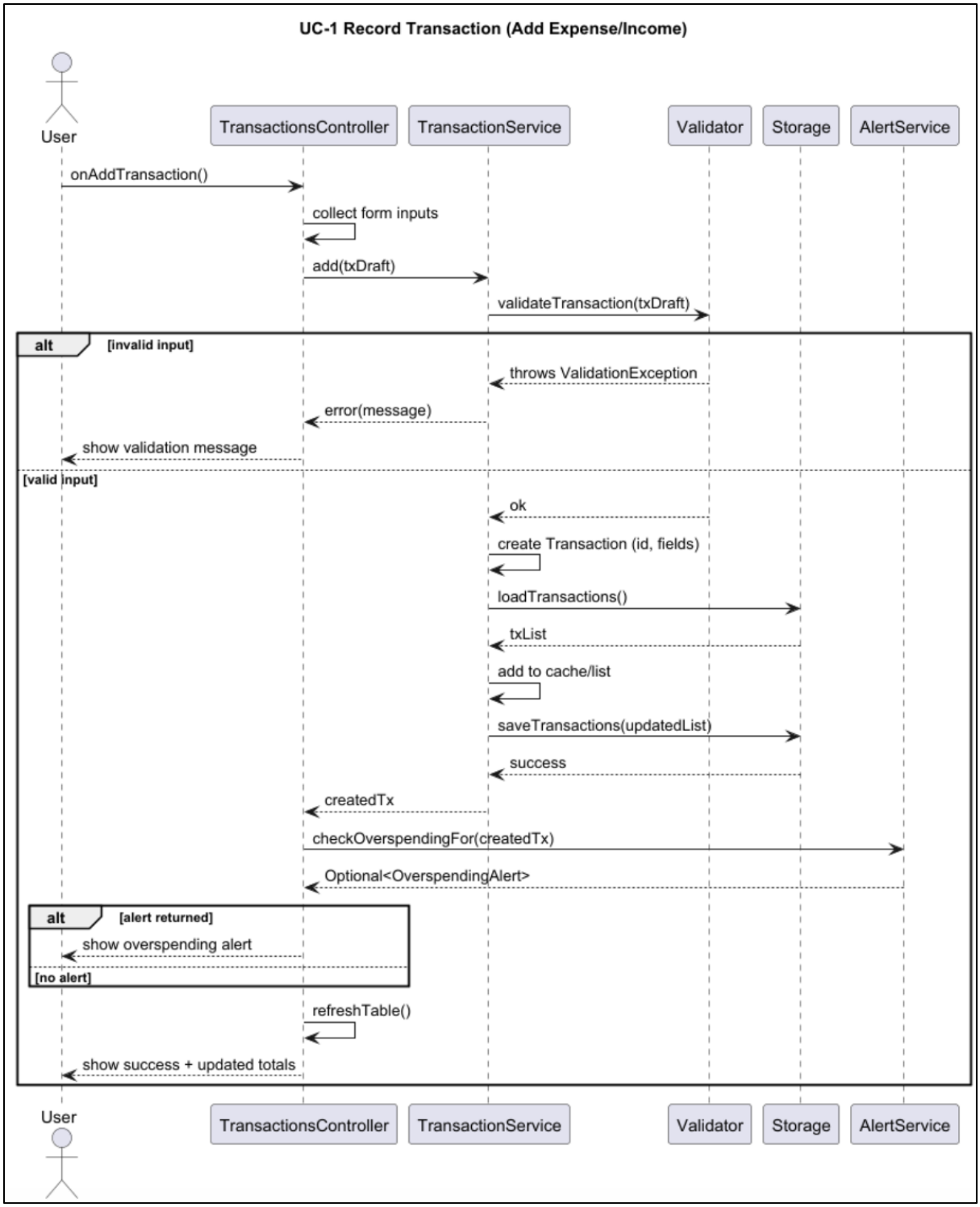


Figure 3-6: Sequence Diagram — UC-1 Record Transaction

3.3.2 SD-2 — UC-4 Set Monthly Budget + UC-6 Overspending Alert Check

This sequence diagram shows how a budget is created or updated for a given month and category. After validation, the budget is saved/updated through the service. The flow then

triggers overspending checks by comparing spending (from transactions) against the budget limit and returning an alert result to the UI if needed.

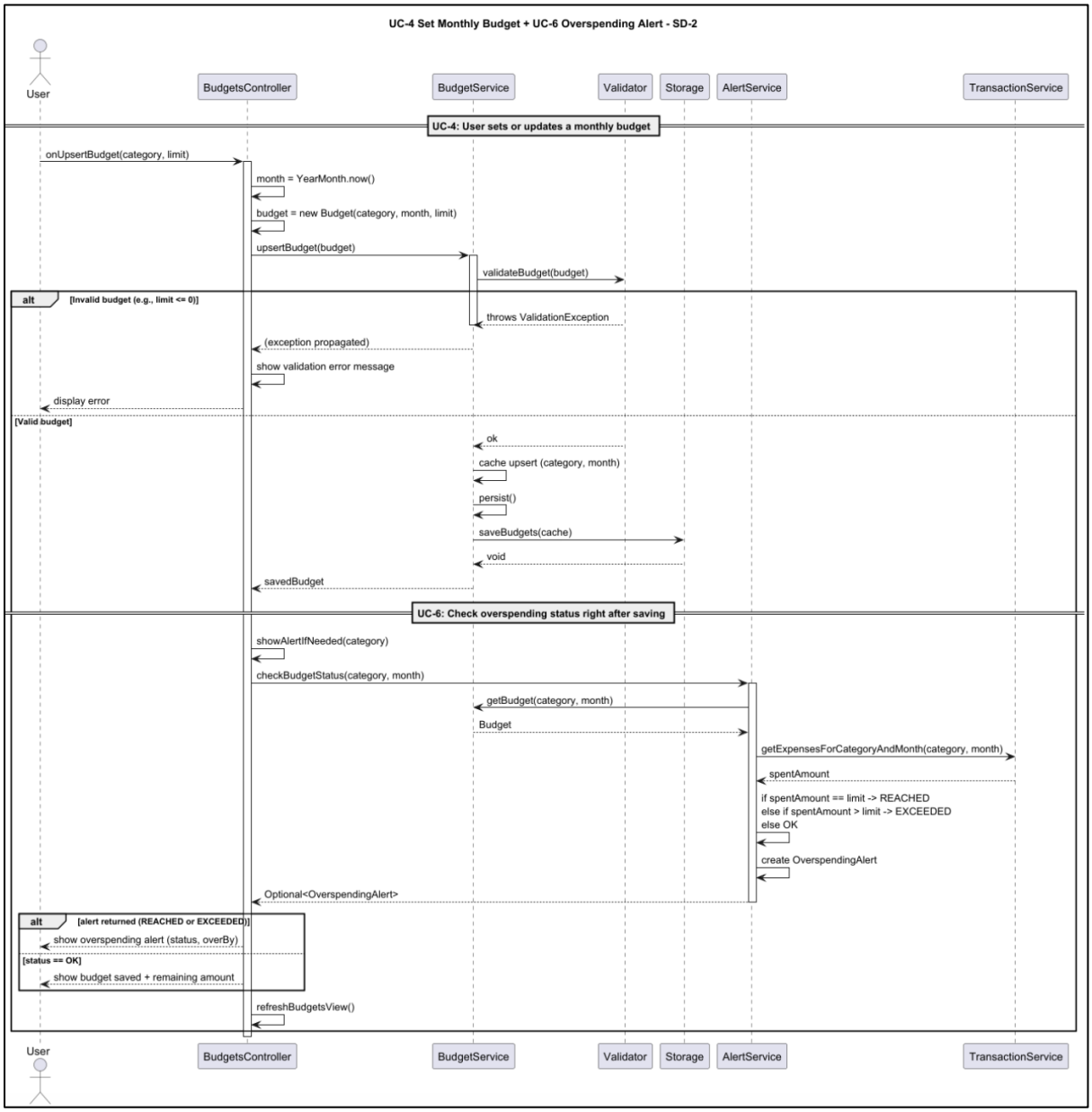


Figure 3-7: Sequence Diagram — UC-4 Set Monthly Budget + UC-6 Alert Check

3.3.3 SD-3 — UC-5 View Summary Report

This sequence diagram shows how a user generates a summary report for a selected time range. The controller creates a *PeriodRange*, calls *ReportService*, and the report service retrieves filtered transactions through *TransactionService*. The output is returned as a report result object that the UI can display.

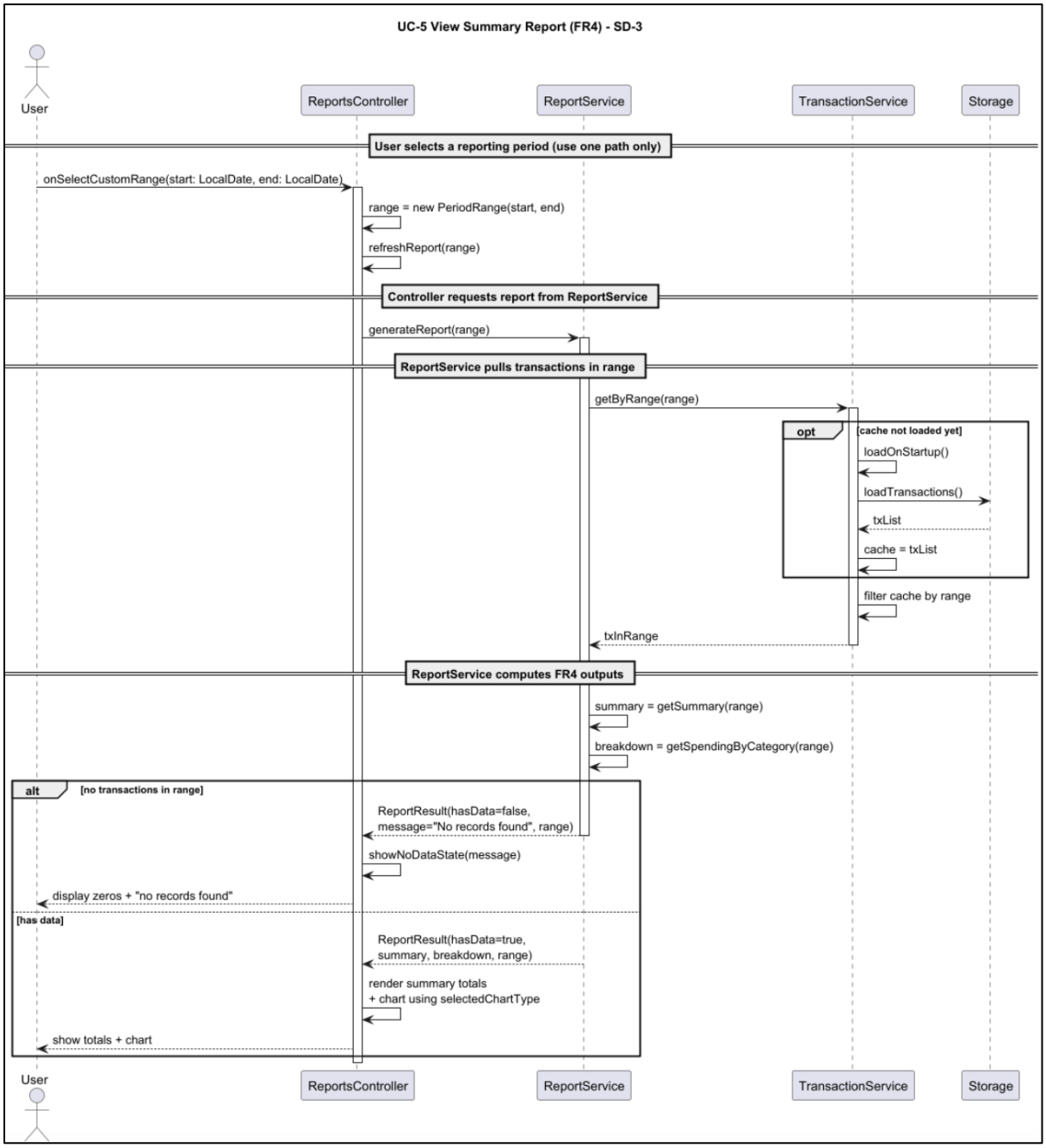


Figure 3-8: Sequence Diagram — UC-5 View Summary Report

3.4 Activity Diagrams

This section provides the activity diagrams for SmartBudget. These diagrams describe the control flow of major user workflows, including actions, decision points, and alternate paths such as invalid inputs or cancellation. The activity diagrams complement the sequence diagrams by focusing on the overall process logic rather than object-to-object messages. Together, they show how the application guides the user through tasks like recording or

amending transactions, setting and updating budgets, generating reports, and initializing the system on startup.

3.4.1 UC-1 Record Transaction (Add)

This activity diagram shows how a user opens the add-transaction modal, enters transaction data, and submits it. It also includes decision points for canceling the modal and handling invalid input before saving.

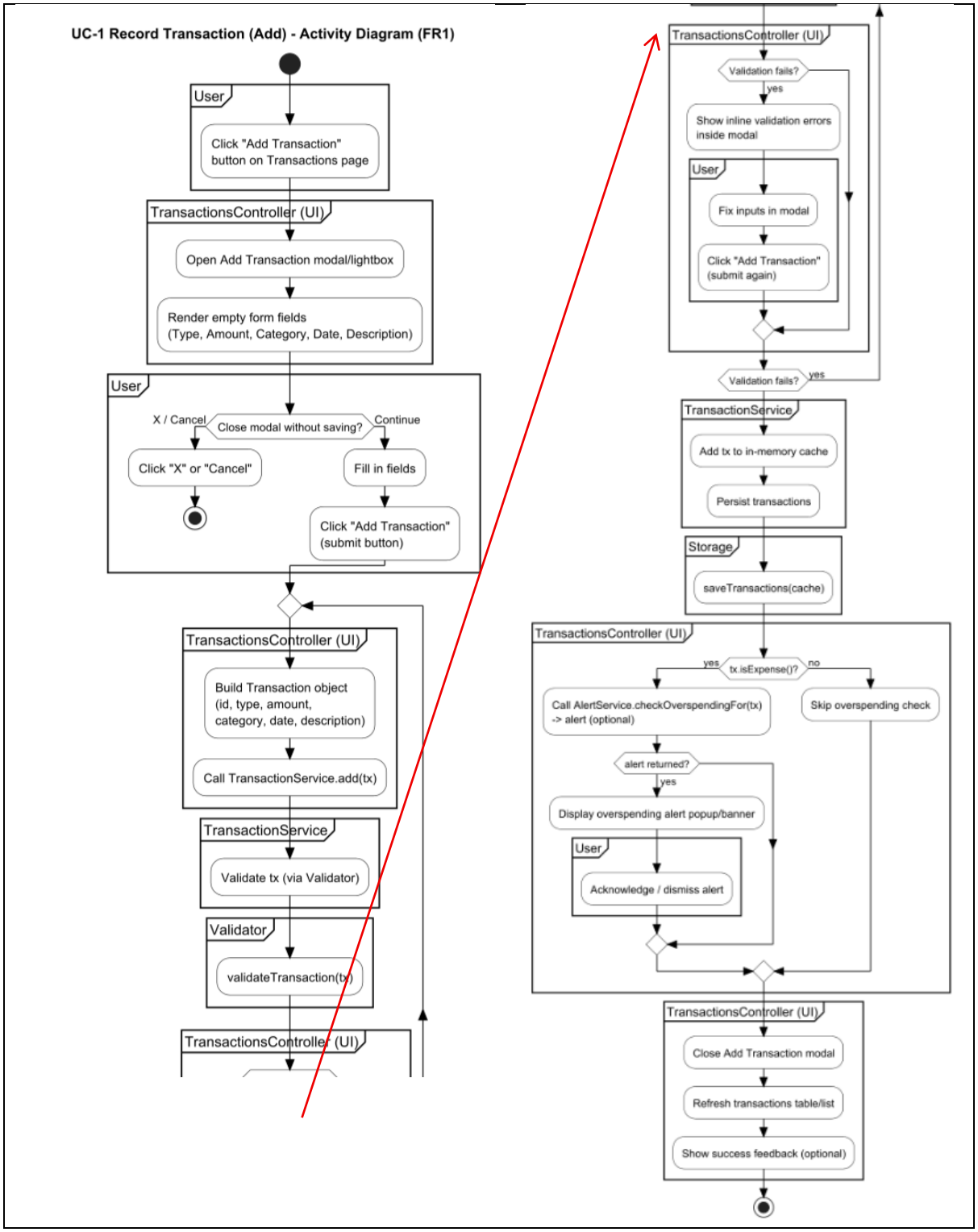


Figure 3-9: Record Transaction (UC-1 FR1).puml screenshot Figure 6.1: Activity Diagram — UC-1 Record Transaction (Add)

3.4.2 UC-2 Amend Transaction

This activity diagram shows editing an existing transaction, validating the updated fields, and saving changes. It also covers cancellation and validation failure paths.

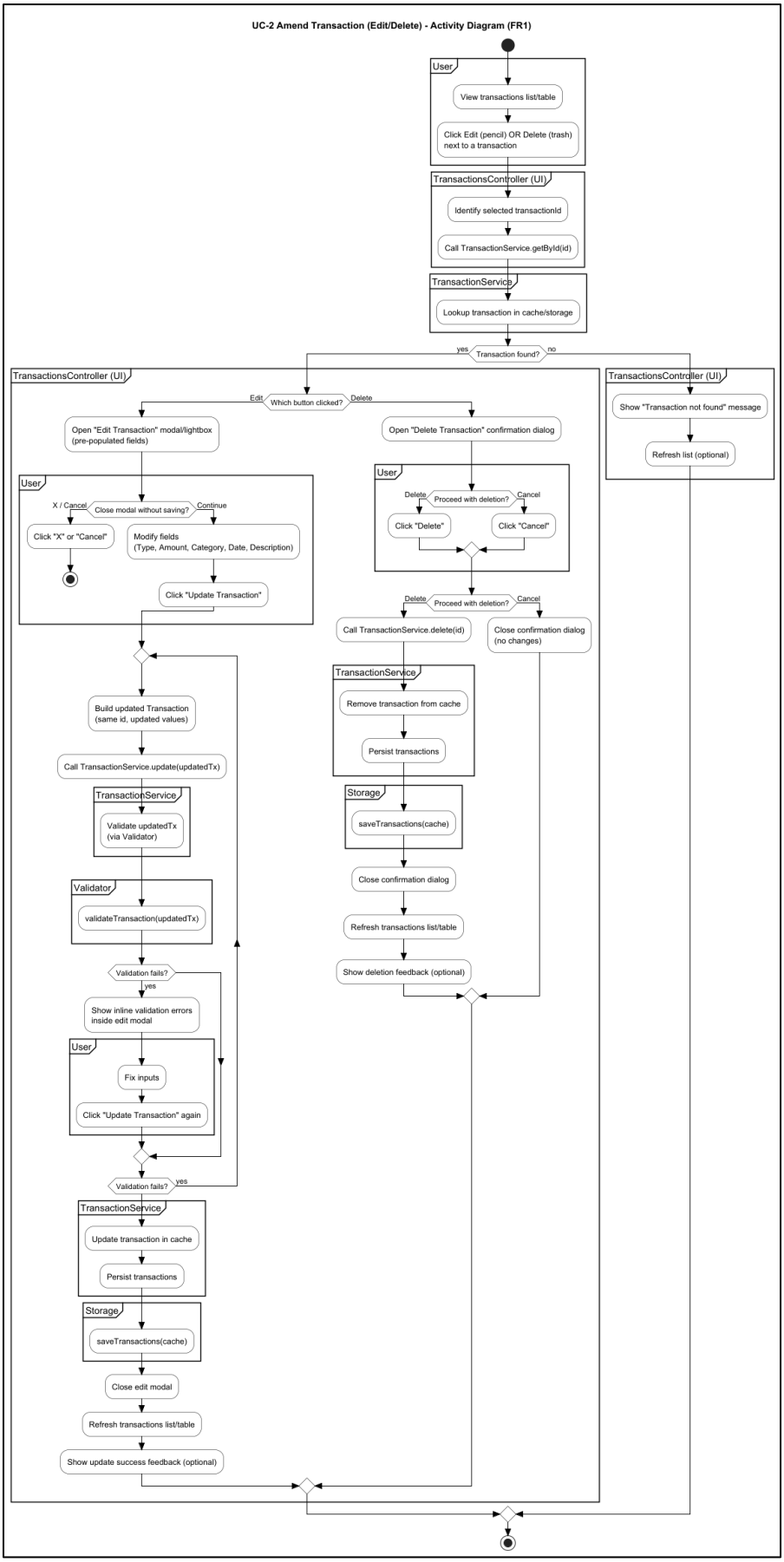


Figure 3-10: Amend Transaction (UC-2 FR1

3.4.3 UC-3 View & Filter Transactions

This activity diagram shows how the user applies filters (such as category/date/type) and how the system refreshes the displayed list based on filter inputs. It also reflects the workflow for clearing or adjusting filters.

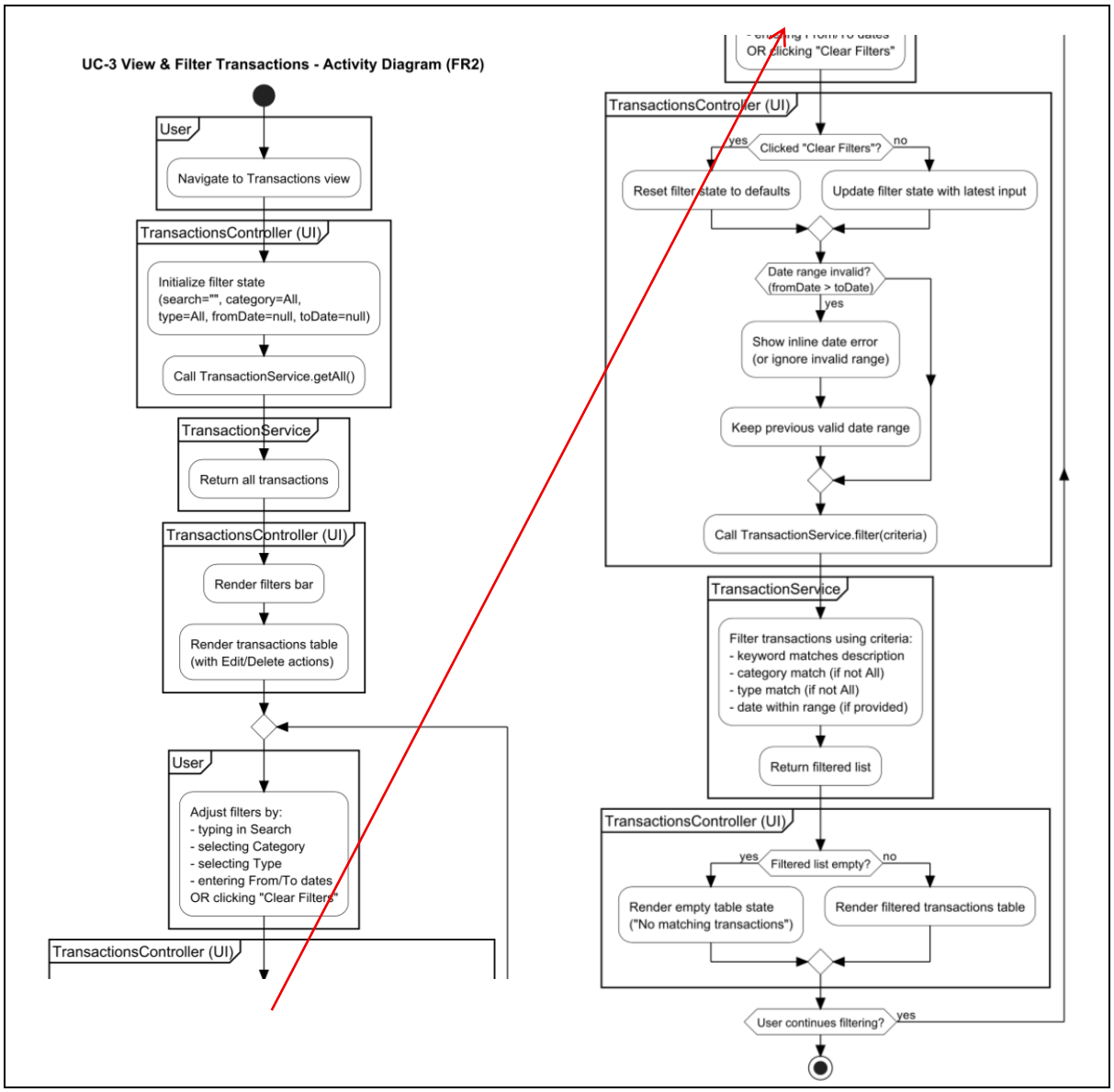


Figure 3-11: Activity Diagram — UC-3 View & Filter Transactions

3.4.4 UC-4 Set Monthly Budget

This activity diagram shows entering a monthly limit for a category, validating inputs, saving the budget, and triggering overspending checks when applicable.

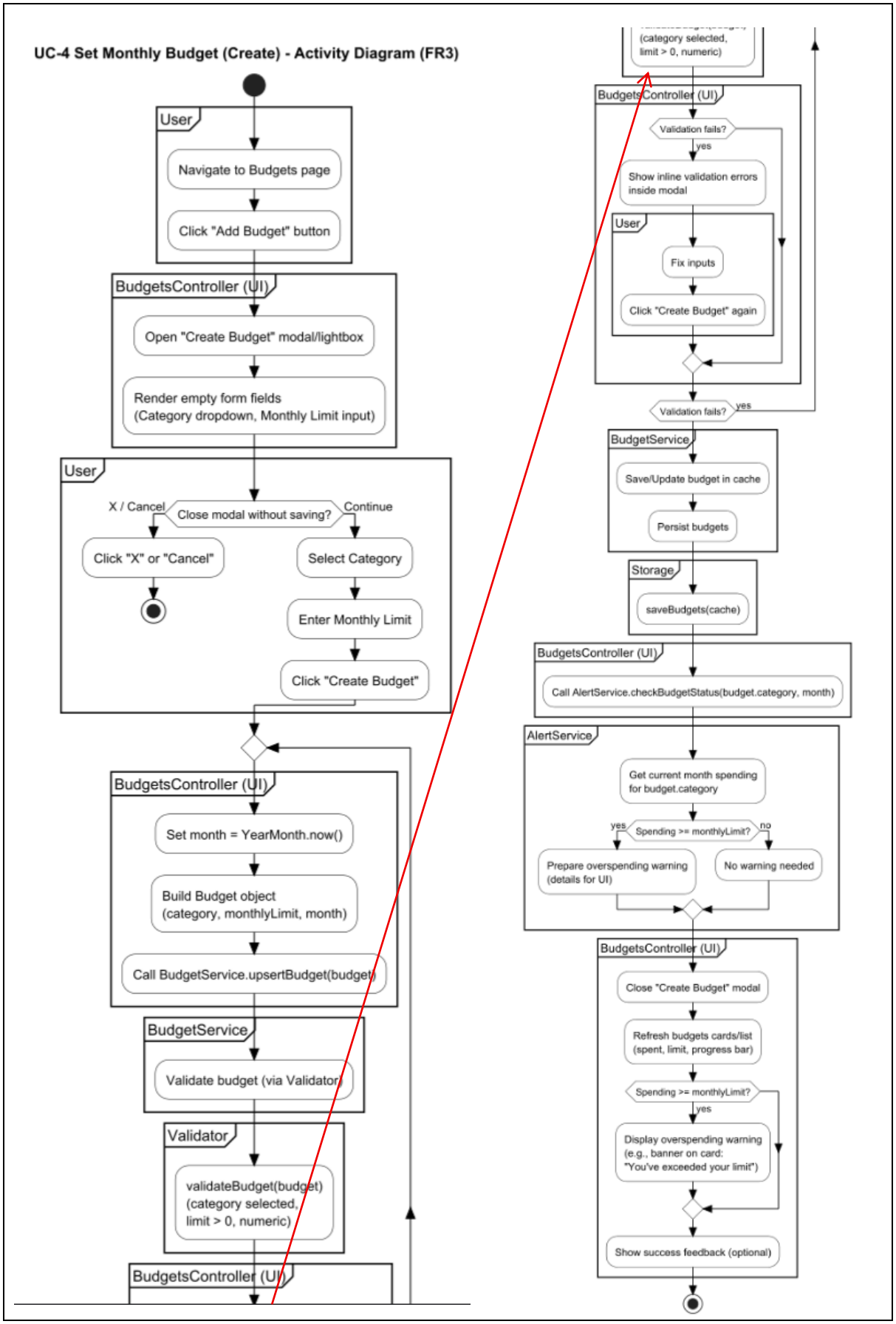


Figure 3-12: Activity Diagram — UC-4 Set Monthly Budget

3.4.5 UC-4 Edit or Delete Monthly Budget

This activity diagram shows the workflow for updating an existing budget or deleting it. It includes decision points for edit vs delete, confirmation steps, and validation errors.

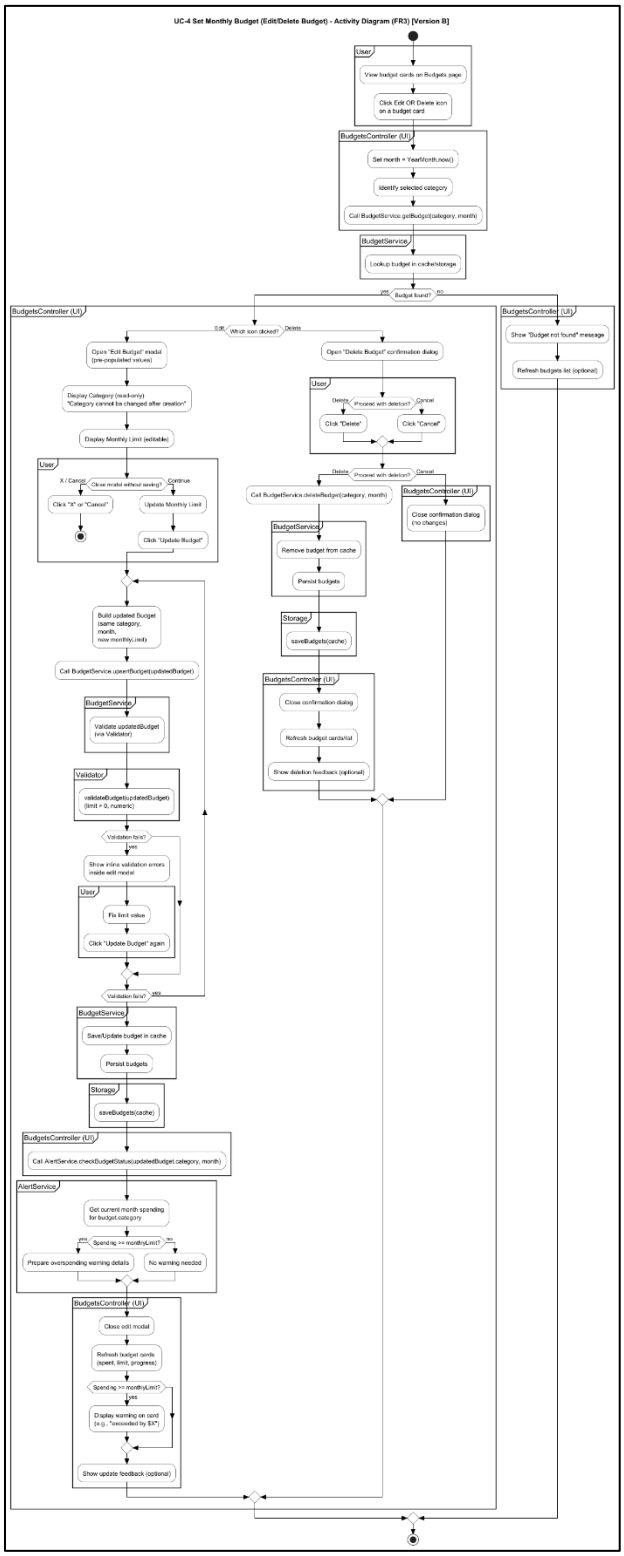


Figure 3-13: UC-4 Edit/Delete Monthly Budget

3.4.6 UC-5 View Summary Report

This activity diagram shows choosing a report period, generating the report, and displaying category totals and summary results. It covers alternate paths such as empty results or invalid ranges (if applicable in your logic).

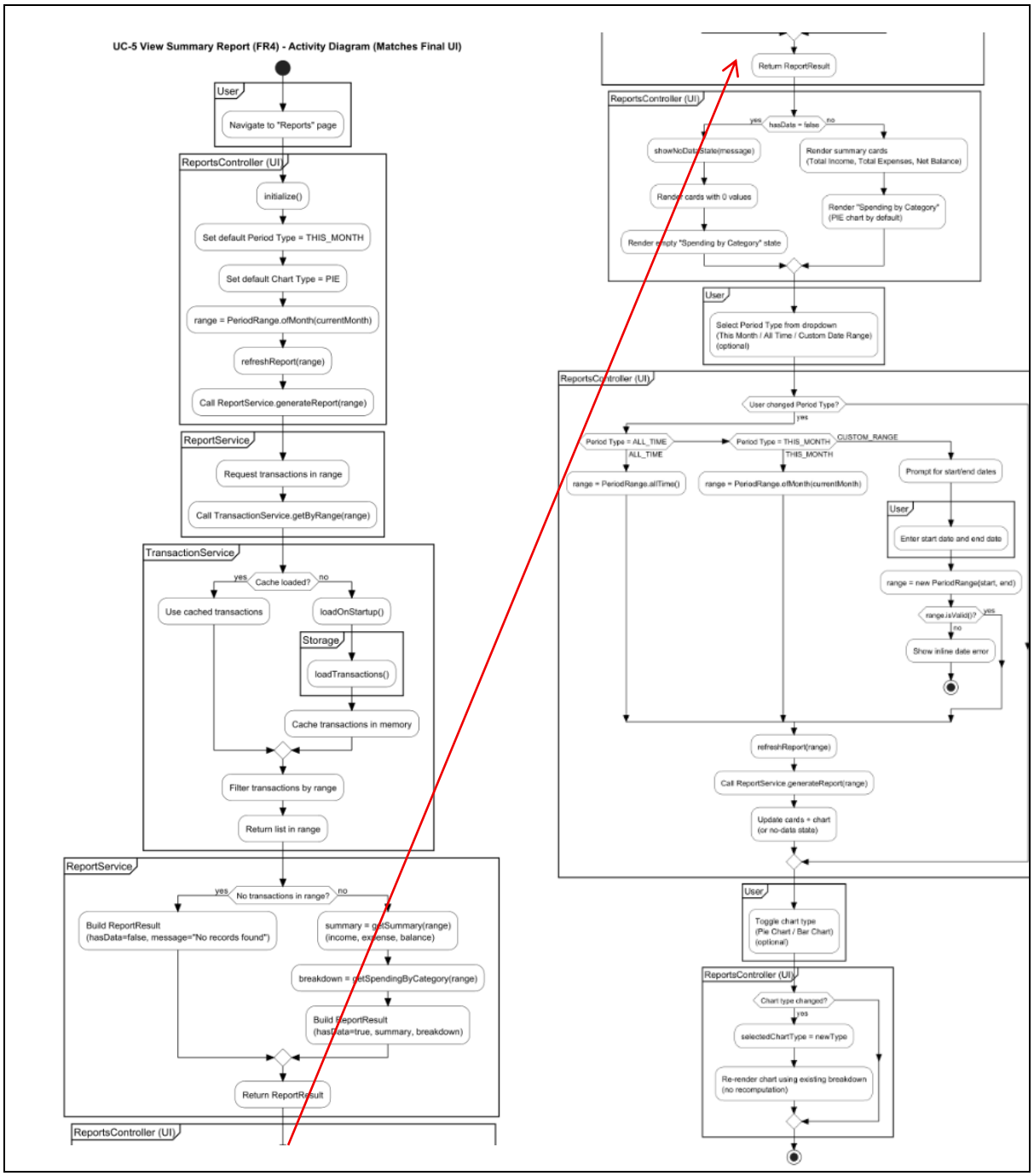


Figure 3-14: Activity Diagram — UC-5 View Summary Report

3.4.7 Load Saved Data on Startup

This activity diagram shows the startup flow of the application: initialization of controllers/services, calling the startup controller to load saved data, and landing on the Transactions page. This workflow supports the planned persistence behavior (even if persistence is completed in the next deliverable).

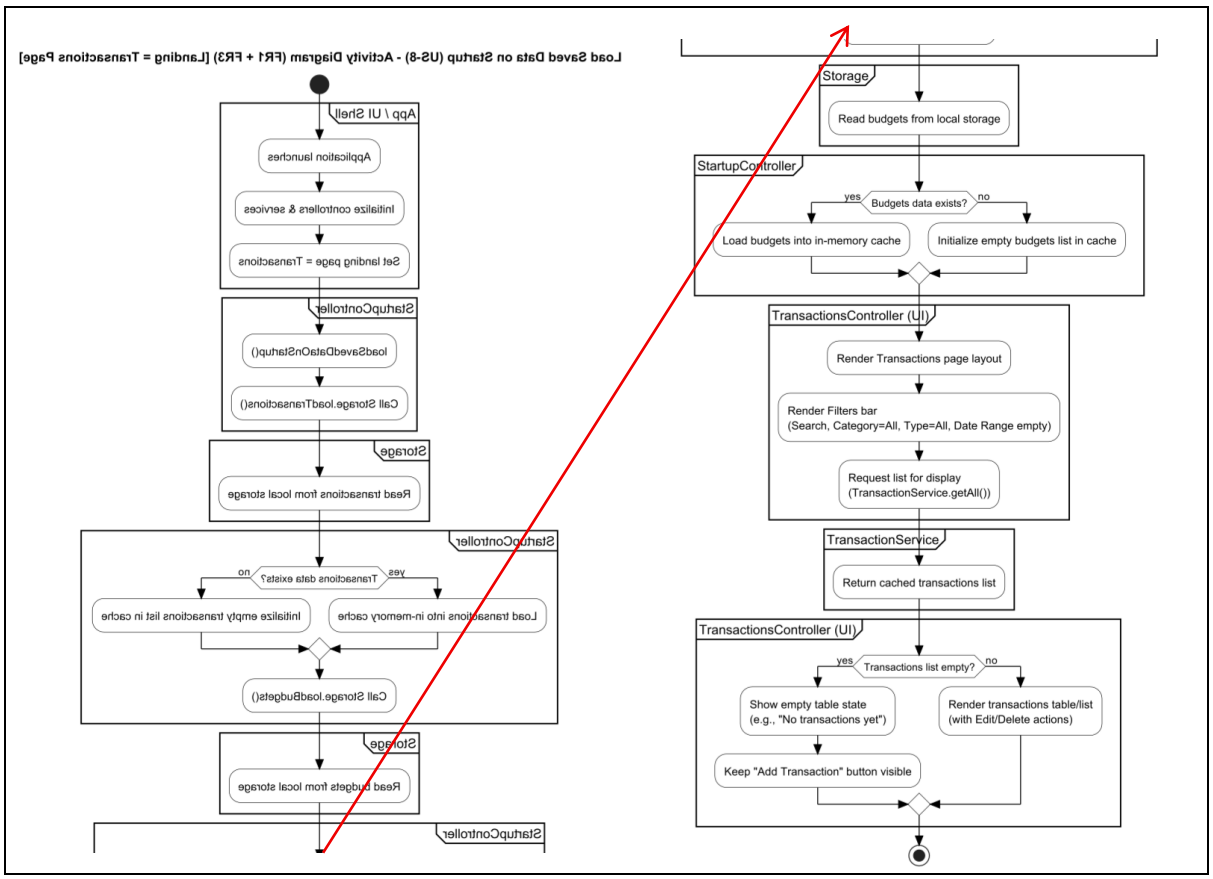


Figure 3-15: Activity Diagram — Load Saved Data on Startup

4 Final Testing Documentation

This chapter presents the final testing documentation for SmartBudget – Personal Finance Tracker. The purpose of this chapter is to show how the completed system was validated and to provide clear evidence of its correctness, reliability, and overall readiness as a finished software product. This chapter brings together the testing methodology used for the project, the test cases prepared and executed for the main system features, the resulting outcomes, and the available coverage evidence in one cohesive testing document. Building on the earlier testing plan and the later implementation-stage testing work, this chapter explains how SmartBudget was tested at both feature level and system level, including the validation of

transactions, budgets, overspending alerts, reports, persistence, and integrated workflows across the application.

4.1 Testing Objectives and Scope

The main goal of testing in the SmartBudget project was to verify that the system works as expected and satisfies the requirements defined for its core features. The testing effort focused on the major functional areas of the application, including recording, editing, and deleting transactions, viewing and filtering transactions, setting monthly budgets, receiving overspending alerts, and generating summary reports. The purpose of this testing work was not only to confirm correctness at feature level, but also to ensure that the overall testing process remained organized, systematic, and clearly connected to the documented requirements of the system.

As the project moved forward, the scope of testing expanded from early test planning into practical validation of the implemented software. The testing work included preparation of structured test cases and traceability between requirements and tests, as well as validation of the running system through manual GUI smoke testing and automated unit testing of key business logic. In addition, coverage analysis was used to show how much of the implemented code was exercised by the automated tests and to identify areas that required stronger validation. Together, these testing activities were intended to provide a clear basis for evaluating the correctness, reliability, and readiness of SmartBudget as a completed software system.

4.2 Testing Methodology and Execution

The testing methodology used for SmartBudget combined structured test planning with practical validation of the implemented system. In the earlier stage of the project, the testing work focused on preparing a clear unit-testing-based plan by defining test cases and linking them to the system requirements. This provided an organized foundation for later validation and ensured that the important features of the system were prepared for systematic testing. As development progressed, the testing approach was expanded beyond planning into actual execution so that both the logic of the system and the main user workflows could be validated in practice.

Testing execution in SmartBudget included three main parts. First, manual GUI smoke testing was used to verify that the main interface workflows behaved correctly during normal use. These checks covered actions such as adding, editing, and deleting transactions, applying filters, managing budgets, generating reports, and navigating between the main pages of the application. Second, automated unit testing was used to validate important business rules in the domain and service layers independently from the GUI. This helped confirm that the core logic of the system behaved correctly under expected conditions. Third, JaCoCo coverage analysis was used to measure how much of the implemented code was exercised by the automated tests and to identify areas where stronger validation was still needed. Together, these methods provided a balanced testing process that examined SmartBudget from both a user workflow perspective and an internal logic perspective.

In practice, test execution was carried out by running the prepared test cases against the implemented features of the system and reviewing the observed outcomes against the expected behavior. Manual GUI testing was used to confirm visible behavior and interaction flow, while automated unit tests were executed to check correctness at code level. The combination of planned test design, executed tests, and coverage analysis helped keep the validation process organized and provided meaningful evidence of the system's correctness, reliability, and readiness as a completed software product.

4.3 Test Cases and Traceability

To keep the testing process organized and complete, the SmartBudget project used a traceability-based testing structure in which each major requirement was linked to one or more planned test cases. This approach helped ensure that the important features of the system were not left untested and that the relationship between requirements and validation remained clear throughout the project. The test planning work prepared a structured foundation for later execution by defining how each requirement would be checked and by documenting the expected behavior of the system in a consistent way.

Table 1: Requirement-to-Test Case Traceability Matrix

Requirement ID	Requirement Description	Test Case ID	Test Case Description	Coverage
FR-001	Record, edit, delete income/expense transactions; persist locally across sessions	TC-001	Add expense with valid inputs (amount, date, category, optional note)	Fully Covered
FR-001	— same as above —	TC-002	Add income with valid inputs	Fully Covered
FR-001	— same as above —	TC-003	Reject invalid amount (non-numeric / negative)	Fully Covered
FR-001	— same as above —	TC-004	Reject missing required fields (amount, date, type) with clear feedback	Fully Covered
FR-001	— same as above —	TC-005	Edit an existing transaction and verify totals update	Fully Covered
FR-001	— same as above —	TC-006	Delete transaction with confirmation prompt; totals update	Fully Covered
FR-001	— same as above —	TC-007	Cancel delete preserves the transaction	Fully Covered
FR-001	— same as above —	TC-008	Data persists across app restart (transactions remain available)	Fully Covered
FR-002	View transactions and apply filters (date range, category)	TC-009	View all transactions in list/table	Fully Covered
FR-002	— same as above —	TC-010	Filter by category (single category)	Fully Covered
FR-002	— same as above —	TC-011	Filter by date range (start–end)	Fully Covered
FR-002	— same as above —	TC-012	Combined filters (date range + category) narrow results and update totals	Fully Covered
FR-002	— same as above —	TC-013	No matches → empty results + zero totals shown	Fully Covered

Requirement ID	Requirement Description	Test Case ID	Test Case Description	Coverage
FR-002	— same as above —	TC-014	Clear filters restores full transaction list	Fully Covered
FR-002	— same as above —	TC-015	Keyword search narrows list	Fully Covered
FR-003	Set monthly spending limit per category and alert on reaching/exceeding	TC-016	Set monthly budget (valid positive amount)	Fully Covered
FR-003	— same as above —	TC-017	Reject invalid budget inputs (negative/blank) with feedback	Fully Covered
FR-003	— same as above —	TC-018	Update an existing budget; remaining allowance recalculates	Fully Covered
FR-003	— same as above —	TC-019	Alert triggers when spending equals the limit	Fully Covered
FR-003	— same as above —	TC-020	Alert triggers when spending exceeds the limit; shows overage amount	Fully Covered
FR-003	— same as above —	TC-021	No budgets defined → no alert appears	Fully Covered
FR-004	Provide period summary (income, expenses, balance) + at least one chart	TC-022	Monthly summary totals compute correctly (income, expenses, balance)	Fully Covered
FR-004	— same as above —	TC-023	Custom range summary computes correctly	Fully Covered
FR-004	— same as above —	TC-024	No data in selected period → zeros + “no records found” note	Fully Covered
FR-004	— same as above —	TC-025	Category chart (pie/bar) reflects data accurately with clear labels	Fully Covered

Requirement ID	Requirement Description	Test Case ID	Test Case Description	Coverage
FR-004	— same as above —	TC-026	Chart and summary auto-update when transactions or filters change	Fully Covered
NFR-Usability	Simple, consistent JavaFX UI; clear labels; readable fonts; sensible navigation	TC-027	Heuristic usability review of labels, font sizes, and navigation between Transactions/Budgets/Reports	NA
NFR-Performance	Responsive under typical datasets; quick loads	TC-028	Load 1–5k transactions; ensure UI remains responsive (no noticeable lag on list/filter)	NA
NFR-Reliability	Immediate save to local storage; graceful handling of invalid input with clear feedback	TC-029	Immediate persistence after create/edit/delete; verify storage writes and recovery on restart	NA
NFR-Privacy/Offline	Full offline operation; no network transmission	TC-030	Run with network disabled; verify all functions work and no network calls are made	NA
NFR-Portability	Runs on Windows, macOS, Linux (supported Java runtime)	TC-031	Smoke run on 3 platforms; application starts and core flows (add/view) work	NA
NFR-Maintainability	Modular (e.g., MVC) structure; unit tests for core business logic	TC-032	Project structure review + presence of unit tests for core model/services	NA

The traceability matrix is used to show how the functional and non-functional requirements of SmartBudget are connected to specific test cases. This makes it easier to see what is being tested, how it will be verified, and whether all major areas of the system are covered. In addition to requirement-level traceability, the testing documentation also maps test cases to the project's use cases and user stories so that validation can be understood from both technical and user-centered perspectives. Together, these mappings strengthen the completeness of the testing documentation and show that the planned tests reflect the real workflows expected in the system.

Table 2: Mapping Test Cases to Use Cases

Use Case ID	Use Case Goal	Test Case ID(s)	Coverage
UC-1: Record Transaction	Add income/expense and update totals	TC-001, TC-002, TC-003, TC-004, TC-022, TC-026	Fully Covered
UC-2: Amend Transaction	Modify or delete an existing entry; refresh totals	TC-005, TC-006, TC-007, TC-022, TC-026	Fully Covered
UC-3: View & Filter Transactions	Review by period/category; empty results path	TC-009, TC-010, TC-011, TC-012, TC-013, TC-014, TC-015	Fully Covered
UC-4: Set Monthly Budget	Define/adjust per-category limit	TC-016, TC-017, TC-018	Fully Covered
UC-5: View Summary Report	Select period; show totals and chart; no-data path	TC-022, TC-023, TC-024, TC-025	Fully Covered
UC-6: Receive Overspending Alert	Notify on reach/exceed budget; precondition = budgets defined	TC-019, TC-020, TC-021	Fully Covered

Table 3: Mapping Test Cases to User Stories

Use Story ID	Story Summary	Related FR	Test Case ID(s)	Coverage
US-1: Record a Transaction	Add expense or income; immediate save; totals update; success feedback	FR-001	TC-001, TC-002, TC-004, TC-022	Fully Covered
US-2: Edit or Delete a Transaction	Edit/delete; confirmation; totals/summaries update	FR-001	TC-005, TC-006, TC-007, TC-022	Fully Covered
US-3: View & Filter Transactions	View all; filter by category/date/keyword; clear filters	FR-002	TC-009, TC-010, TC-011, TC-012, TC-014, TC-015	Fully Covered
US-4: Set a Monthly Budget	Set/update budget; show remaining; reject invalid	FR-003	TC-016, TC-017, TC-018	Fully Covered
US-5: Receive Overspending Alerts	Trigger at ≥ limit; show category and overage; remains visible	FR-003	TC-019, TC-020, TC-021	Fully Covered

US-6: View Monthly Summary	Select period; totals computed; clear message for no data	FR-004	TC-022, TC-024	Fully Covered
US-7: View Spending Chart	Present chart; auto-updates; clear labels/legend	FR-004	TC-025, TC-026	Fully Covered
US-8: Load Saved Data on Startup	Auto-load transactions/budgets at startup; notify if none	FR-001 / FR-003	TC-008	Fully Covered

The detailed test case descriptions further support this traceability by defining each test in a structured format, including its ID, title, related requirement, preconditions, test steps, expected result, and status field. These descriptions provide a clear reference for how each feature should be validated and help keep the testing process systematic and repeatable. In the final report, these test cases serve as the documented basis for the testing work carried out on SmartBudget and show that the project's validation process was planned in a detailed and organized way.

4.3.1 TC-001

Table 4: TC-01 test case description

Test Case ID	**TC-01**
Title	Add Expense – Valid Inputs
Requirement	FR-001; UC-1; US-1
Preconditions	App is running; "Transactions" view available
Steps	1. Open Transactions; Click "Add Transaction"; Select Expense. 2. Enter valid amount (e.g., 25.50), pick a valid date, choose category (e.g., Food), add optional note. 3. Click "Save".
Expected Result	Expense row added; expense increases and balance totals decreases; success feedback; persisted.
Actual Result	Expense row added; expense increases and balance totals decreases; success feedback; persisted.
Status	Pass
Priority	High

4.3.2 TC-002

Table 5: TC-02 test case description

Test Case ID	**TC-02**
Title	Add Income – Valid Inputs
Requirement	FR-001; UC-1; US-1
Preconditions	App is running; “Transactions” view available
Steps	1. Open Transactions; Click “Add Transaction”; Select Income. 2. Enter valid amount (e.g., 25.50), pick a valid date, choose category (e.g., Salary), add optional note. 3. Click “Save”
Expected Result	Income row added; income and balance totals increase; success feedback; persisted.
Actual Result	Income row added; income and balance totals increase; success feedback; persisted.
Status	Pass
Priority	High

4.3.3 TC-003

Table 6: TC-03 test case description

Test Case ID	**TC-03**
Title	Add Transaction – Reject Invalid Amount
Requirement	FR-001; UC-1; US-1
Preconditions	App is running; “Transactions” view available
Steps	1. Open Transactions; Click “Add Transaction”; Select Income/Expense. 2. Enter non-numeric (e.g., “abc”) or negative amount. 3. Click “Save”.
Expected Result	Save blocked; clear validation message; no record created; no totals change.
Actual Result	Save blocked; clear validation message; no record created; no totals change.
Status	Pass
Priority	High

4.3.4 TC-004

Table 7: TC-04 test case description

Test Case ID	**TC-04**
Title	Add Transaction – Reject Missing Required Fields
Requirement	FR-001; UC-1; US-1
Preconditions	App is running; “Transactions” view available
Steps	1. Open Transactions; Click “Add Transaction”; Select Income/Expense. 2. Leave amount or date or type empty. 3. Click “Save”.
Expected Result	Save blocked; specific error message; no changes persisted.
Actual Result	Save blocked; specific error message; no changes persisted.
Status	Pass
Priority	High

4.3.5 TC-005

Table 8: TC-05 test case description

Test Case ID	**TC-05**
Title	Edit Transaction – Totals Recalculate
Requirement	FR-001; UC-2; US-2
Preconditions	At least one existing transaction
Steps	1. Open an existing transaction. 2. Change amount (e.g., 20 → 30). 3. Click “Save”.
Expected Result	Row updates; affected totals/summary recalculates; success feedback; persisted.
Actual Result	Row updates; affected totals/summary recalculates; success feedback; persisted.
Status	Pass
Priority	High

4.3.6 TC-006

Table 9: TC-06 test case description

Test Case ID	**TC-06**
Title	Delete Transaction – Confirmation and Totals Update
Requirement	FR-001; UC-2; US-2
Preconditions	At least one existing transaction
Steps	1. Open an existing transaction. 2. Click delete. 3. Confirm deletion.
Expected Result	Row removed; totals/summary recalculates; confirmation used to prevent accidental deletion; persisted removal.
Actual Result	Row removed; totals/summary recalculates; confirmation used to prevent accidental deletion; persisted removal.
Status	Pass
Priority	High

4.3.7 TC-007

Table 10: TC-07 test case description

Test Case ID	**TC-07**
Title	Delete Transaction – Cancel Preservation
Requirement	FR-001; UC-2; US-2
Preconditions	At least one existing transaction
Steps	1. Open an existing transaction. 2. Click delete. 3. In confirmation, click Cance
Expected Result	No deletion; list and totals unchanged
Actual Result	No deletion; list and totals unchanged
Status	Pass
Priority	Medium

4.3.8 TC-008

Table 11: TC-08 test case description

Test Case ID	**TC-08**
Title	Persistence – Transactions Survive App Restart

Requirement	FR-001; US-8
Preconditions	At least one existing transaction
Steps	1. Close app. 2. Reopen app. 3. Go to Transactions.
Expected Result	Previously saved transactions load automatically; no manual import required.
Actual Result	Previously saved transactions load automatically; no manual import required.
Status	Pass
Priority	High

4.3.9 TC-009

Table 12: TC-09 test case description

Test Case ID	**TC-09**
Title	View All Transactions – Default List/Table
Requirement	FR-002; UC-3; US-3
Preconditions	At least one existing transaction
Steps	1. Open Transactions view.
Expected Result	All transactions visible in a readable table/list with key columns; totals visible.
Actual Result	All transactions visible in a readable table/list with key columns; totals visible.
Status	Pass
Priority	High

4.3.10 TC-010

Table 13: TC-10 test case description

Test Case ID	**TC-10**
Title	Filter by Category (Single Category)
Requirement	FR-002; UC-3; US-3
Preconditions	Transactions exist across multiple categories.
Steps	1. Apply category filter (e.g., Food).

Expected Result	Only rows with category = “Food” shown; totals/summary reflect filtered subset.
Actual Result	Only rows with category = “Food” shown; totals/summary reflect filtered subset.
Status	Pass
Priority	High

4.3.11 TC-011

Table 14: TC-11 test case description

Test Case ID	**TC-11**
Title	Filter by Date Range
Requirement	FR-002; UC-3; US-3
Preconditions	Transactions exist across multiple dates.
Steps	1. Set start and end date.
Expected Result	Only transactions within range shown; totals reflect range.
Actual Result	Only transactions within range shown; totals reflect range.
Status	Pass
Priority	High

4.3.12 TC-012

Table 15: TC-12 test case description

Test Case ID	**TC-12**
Title	Combined Filters – Date Range + Category
Requirement	FR-002; UC-3; US-3
Preconditions	Data spans categories and dates.
Steps	1. Apply date range. 2. Apply category.
Expected Result	List narrows to intersection; totals update accordingly.
Actual Result	List narrows to intersection; totals update accordingly.
Status	Pass
Priority	High

4.3.13 TC-013

Table 16: TC-13 test case description

Test Case ID	**TC-13**
Title	Filter results – No Matches
Requirement	FR-002; UC-3; US-3
Preconditions	Data set present
Steps	1. Apply filters that yield zero results.
Expected Result	Empty list message + zero totals displayed clearly.
Actual Result	Empty list message + zero totals displayed clearly.
Status	Pass
Priority	Medium

4.3.14 TC-014

Table 17: TC-14 test case description

Test Case ID	**TC-14**
Title	Clear Filters – Restore Full List
Requirement	FR-002; UC-3; US-3
Preconditions	Active filter(s) applied.
Steps	1. Click “Clear Filters”
Expected Result	All transactions visible again; totals reflect full data set
Actual Result	All transactions visible again; totals reflect full data set
Status	Pass
Priority	Medium

4.3.15 TC-015

Table 18: TC-15 test case description

Test Case ID	**TC-15**
Title	Keyword Search – Narrows List
Requirement	FR-002; UC-3; US-3
Preconditions	Transactions with varied notes/payee/category names exist
Steps	1. Enter keyword (e.g., “coffee”) in search.
Expected Result	Only matching rows shown (by configured fields: note, payee, category); totals update; clearing search restores list.

Actual Result	Only matching rows shown (by configured fields: note, payee, category); totals update; clearing search restores list.
Status	Pass
Priority	Medium

4.3.16 TC-016

Table 19: TC-16 test case description

Test Case ID	**TC-16**
Title	Set Monthly Budget – Valid Positive Amount
Requirement	FR-003; UC-4; US-4
Preconditions	Category exists (e.g., Food).
Steps	1. Open Budgets. 2. Select “Food”. 3. Enter 200.00. 4. Save
Expected Result	Budget stored for current month; remaining allowance visible.
Actual Result	Budget stored for current month; remaining allowance visible.
Status	Pass
Priority	Medium

4.3.17 TC-017

Table 20: TC-17 test case description

Test Case ID	**TC-17**
Title	Set Monthly Budget – Reject Invalid Inputs
Requirement	FR-003; UC-4; US-4
Preconditions	Budgets view open
Steps	1. Enter negative or blank budget. 2. Save
Expected Result	Save blocked; clear error message; no budget created/changed
Actual Result	Save blocked; clear error message; no budget created/changed
Status	Pass
Priority	Medium

4.3.18 TC-018

Table 21: TC-18 test case description

Test Case ID	TC-18
Title	Update Existing Budget – Recalculate Remaining
Requirement	FR-003; UC-4; US-4
Preconditions	Budget already set for category; some spending exists
Steps	1. Edit “Food” budget 200 → 250. 2. Save
Expected Result	New limit stored; remaining allowance recalculated and displayed.
Actual Result	New limit stored; remaining allowance recalculated and displayed.
Status	Pass
Priority	Medium

4.3.19 TC-019

Table 22: TC-19 test case description

Test Case ID	TC-19
Title	Overspending Alert – Trigger at Equal to Limit
Requirement	FR-003; UC-6; US-5
Preconditions	Budget = 100 for Food; current spending = 90
Steps	1. Add Food expense of 10.
Expected Result	Alert appears indicating limit reached (category shown); remains visible until acknowledged/reset
Actual Result	Alert appears indicating limit reached (category shown); remains visible until acknowledged/reset
Status	Pass
Priority	Medium

4.3.20 TC-020

Table 23: TC-20 test case description

Test Case ID	TC-20
Title	Overspending Alert – Trigger when Exceeding Limit
Requirement	FR-003; UC-6; US-5

Preconditions	Budget = 100 for Food; current spending = 95
Steps	1. Add Food expense of 10 (total 105).
Expected Result	Alert shows category and overage amount (e.g., +5); persists until acknowledged/reset
Actual Result	Alert shows category and overage amount (e.g., +5); persists until acknowledged/reset
Status	Pass
Priority	Medium

4.3.21 TC-021

Table 24: TC-21 test case description

Test Case ID	**TC-21**
Title	No Budget Defined – No Alert
Requirement	FR3; UC-6; US-5
Preconditions	No budgets set.
Steps	1. Add multiple expenses in any category.
Expected Result	No budget alert appears.
Actual Result	No budget alert appears.
Status	Pass
Priority	Medium

4.3.22 TC-022

Table 25: TC-22 test case description

Test Case ID	**TC-22**
Title	Monthly Summary – Correct Totals
Requirement	FR-004; UC-5; US-6
Preconditions	Month has known set of incomes/expenses
Steps	1. Open Reports. 2. Select “This Month”.
Expected Result	Income, expenses, and balance compute correctly and match dataset.
Actual Result	Income, expenses, and balance compute correctly and match dataset.

Status	Pass
Priority	High

4.3.23 TC-023

Table 26: TC-23 test case description

Test Case ID	**TC-23**
Title	Custom Range Summary – Correct Totals
Requirement	FR-004; UC-5; US-6
Preconditions	Data across multiple months
Steps	1. Choose custom start/end dates across months.
Expected Result	Totals reflect only the selected range; displayed clearly.
Actual Result	Totals reflect only the selected range; displayed clearly.
Status	Pass
Priority	Medium

4.3.24 TC-024

Table 27: TC-24 test case description

Test Case ID	**TC-24**
Title	Summary – No Data Path
Requirement	FR-004; UC-5; US-7
Preconditions	Select a period with no transactions
Steps	1. Open Reports
Expected Result	Zeros shown for totals; clear “no records found” note
Actual Result	Zeros shown for totals; clear “no records found” note
Status	Pass
Priority	Medium

4.3.25 TC-025

Table 28: TC-25 test case description

Test Case ID	**TC-25**
Title	Category Chart – Accurate Visualization
Requirement	FR-004; UC-5; US-7

Preconditions	Transactions across multiple categories.
Steps	1. Open Reports with chart (pie/bar).
Expected Result	Chart values/percentages match underlying totals; labels/legend are clear.
Actual Result	Chart values/percentages match underlying totals; labels/legend are clear.
Status	Pass
Priority	Low

4.3.26 TC-026

Table 29: TC-26 test case description

Test Case ID	**TC-26**
Title	Reports/Charts Auto-Update on Data/Filter Change
Requirement	FR-004; UC-5; US-7
Preconditions	Reports open with visible chart; baseline totals noted
Steps	1. Add/edit/delete a transaction OR change filters
Expected Result	Summary and chart refresh immediately to reflect the change.
Actual Result	Summary and chart refresh immediately to reflect the change.
Status	Pass
Priority	Medium

4.3.27 TC-027

Table 30: TC-27 test case description

Test Case ID	**TC-27**
Title	Usability Heuristics – Labels, Fonts, Navigation
Requirement	NFR-Usability
Preconditions	UI available (Transactions, Budgets, Reports)
Steps	1. Inspect labels, font sizes, contrast. 2. Walk through navigation paths between key views.
Expected Result	Clear labels; readable fonts; predictable navigation; consistent UI components
Actual Result	Clear labels; readable fonts; predictable navigation; consistent UI components

Status	Pass
Priority	Medium

4.3.28 TC-028

Table 31: TC-28 test case description

Test Case ID	**TC-28**
Title	Performance – Typical Dataset Responsiveness
Requirement	NFR- Performance
Preconditions	Seed transactions locally
Steps	1. Launch app and open Transactions. 2. Apply filters/search repeatedly
Expected Result	Loads and interactions complete without noticeable lag; list/filters remain responsive
Actual Result	Loads and interactions complete without noticeable lag; list/filters remain responsive
Status	Pass
Priority	Medium

4.3.29 TC-029

Table 32: TC-29 test case description

Test Case ID	**TC-29**
Title	Reliability – Immediate Save & Graceful Invalid Handling
Requirement	NFR- Reliability
Preconditions	App running
Steps	1. Add/edit/delete; immediately kill and relaunch app. 2. Attempt invalid inputs on add/edit.
Expected Result	Most recent changes present after relaunch (immediate save); invalid inputs yield clear feedback; no data corruption
Actual Result	Most recent changes present after relaunch (immediate save); invalid inputs yield clear feedback; no data corruption
Status	Pass
Priority	High

4.3.30 TC-030

Table 33: TC-30 test case description

Test Case ID	TC-30
Title	Privacy/Offline – No Network Dependency
Requirement	Privacy/Offline
Preconditions	Device offline (disable network)
Steps	1. Launch app. 2. Perform add/view/filter/report flows
Expected Result	All features function offline; no network calls attempted
Actual Result	All features function offline; no network calls attempted
Status	Pass
Priority	High

4.3.31 TC-031

Table 34: TC-31 test case description

Test Case ID	TC-31
Title	Portability – Run on Windows/macOS/Linux
Requirement	NFR-Portability
Preconditions	Build packaged with supported Java runtime
Steps	1. Start app on each OS. 2. Smoke test add/view flows.
Expected Result	App launches and core flows work on all three platforms
Actual Result	App launches and core flows work on all three platforms
Status	Pass
Priority	Medium

4.3.32 TC-032

Table 35: TC-32 test case description

Test Case ID	TC-32
Title	Maintainability – Modular Structure & Unit Tests Present
Requirement	NFR-Maintainability
Preconditions	Repo accessible
Steps	1. Review project structure (e.g., MVC separation.

	2. Verify presence of unit tests for core model/services.
Expected Result	Clear modular separation; baseline unit tests exist; naming/packaging consistent.
Actual Result	Clear modular separation; baseline unit tests exist; naming/packaging consistent.
Status	Pass
Priority	Medium

4.4 Test Results

The executed test results provide practical evidence that the main SmartBudget workflows behaved correctly in the implemented system. Testing results were gathered from both manual GUI validation and automated unit testing so that the project could evaluate the system from two perspectives: the visible behavior of the application during normal use, and the correctness of the underlying business logic independent of the interface. This combination helped confirm that the major features of SmartBudget were working as expected in a running version of the system.

Manual GUI smoke testing was used to verify the main user-facing workflows of the application. These tests confirmed that users could successfully add, edit, and delete transactions, apply filters, create and manage budgets, generate reports for a selected date range, and move correctly between the Transactions, Budgets, and Reports pages. The smoke test results showed passing outcomes for all of these main workflows, providing confidence that the interface and interaction flow were functioning correctly during an active session.

Automated unit testing was used to validate important rules in the domain and service layers. The automated test results showed that 55 unit tests were executed successfully, with all tests passing. These tests mainly covered validation logic and parts of the core business logic, which helped confirm that important internal system behaviors were working correctly even without relying on GUI-based testing. Together with the smoke test results, this provided a stronger overall view of system correctness than either method could provide alone.

4.4.1 Manual GUI Smoke Test Results

The following smoke tests were executed to confirm the GUI operates as expected:

- Add transaction (valid input): Transaction is added and list refreshes — **Pass**
- Add transaction (invalid input): Error shown; transaction not saved — **Pass**
- Edit transaction: Updated values saved and reflected in list — **Pass**

- Delete transaction: Confirmation shown; transaction removed after confirm — **Pass**
- Filtering transactions: Filters update the displayed list correctly — **Pass**
- Add budget (valid input): Budget saved and page refreshes — **Pass**
- Add budget (invalid input): Error shown; budget not saved — **Pass**
- Edit budget / Delete budget: Changes reflected correctly after action — **Pass**
- Generate report for date range: Report output displayed for selected period — **Pass**
- Navigation between pages: User can move between Transactions, Budgets, and Reports pages — **Pass**

4.4.2 Automated Unit Testing Summary

- 55 unit tests were executed successfully (100% passing).
- The unit tests mainly target the validation logic, plus parts of the domain and service layers.
- This provides confidence that key business rules behave correctly independent of the GUI.

4.5 Coverage Summary

Coverage analysis was used to support the validation of SmartBudget by showing how much of the implemented code was exercised through automated unit testing. A JaCoCo report was generated to provide quantitative evidence of automated test execution across the project packages. This coverage evidence complements the manual GUI and smoke testing discussed earlier in this chapter and helps present a broader view of how the final system was validated.

Based on the final JaCoCo report, SmartBudget achieved 24% instruction coverage and 35% branch coverage overall. The report also recorded 337 out of 530 complexity elements, 989 out of 1,400 lines, 122 out of 230 methods, and 22 out of 43 classes as not covered by automated tests. At package level, the strongest automated coverage was observed in the core logic areas of the project, including the service package (89% instruction, 74% branch), domain package (83% instruction, 56% branch), persistence package (80% instruction, 66% branch), and util package (70% instruction, 68% branch). These results show that the main business logic and supporting backend components of the system were exercised effectively through automated testing.

Table 4-36: JaCoCo Coverage Summary (by package)

SmartBudget

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ca.yorku.smartbudget.ui		0%		0%	147	147	562	562	63	63	8	8
ca.yorku.smartbudget.ui.modals		3%		8%	109	118	349	363	47	50	13	14
ca.yorku.smartbudget.service		89%		74%	31	110	13	210	1	33	0	4
ca.yorku.smartbudget.domain		83%		56%	27	100	20	147	4	62	0	11
ca.yorku.smartbudget		0%		n/a	4	4	24	24	4	4	1	1
ca.yorku.smartbudget.util		70%		68%	15	38	13	61	3	11	0	2
ca.yorku.smartbudget.persistence		80%		66%	4	13	8	33	0	7	0	3
Total	5,481 of 7,227	24%	381 of 591	35%	337	530	989	1,400	122	230	22	43

The lower automated coverage values in the UI and UI modal packages should be understood in the context of the project's testing strategy. Since SmartBudget is a JavaFX desktop application, many interface-driven workflows were validated through manual GUI testing, manual smoke testing, and direct workflow execution rather than through unit-test instrumentation. For this reason, the coverage summary should be interpreted together with the earlier manual testing results, which confirmed that the main user-facing features of the application were tested and validated successfully. Together, the automated coverage evidence and the manual testing results provide a complete picture of the system's final validation.

4.6 Testing Conclusion

Overall, the testing work carried out for SmartBudget provides evidence that the major features of the system were validated in a structured and practical way. The testing chapter combines planned test design, requirement traceability, executed GUI workflow checks, automated unit testing, and coverage analysis to present a complete view of the project's validation effort. Together, these testing activities support confidence in the correctness and reliability of the system and show that SmartBudget was tested in a way that supports its final submission as a complete software project.

5 User Manual

This chapter presents the user manual for SmartBudget – Personal Finance Tracker. The purpose of this chapter is to guide end users in understanding, setting up, and using the software effectively through clear and practical instructions. It explains the purpose of the system, the main features available to users, and the steps required to operate the application in everyday use. This chapter also provides guidance on installation and setup, navigation through the main screens, the use of core features such as transactions, budgets, alerts, reports, and saved data handling, as well as troubleshooting support and frequently asked questions. Overall, this

chapter is intended to help users interact with SmartBudget confidently and use its features in a simple, efficient, and reliable way.

5.1 Introduction

SmartBudget is a personal finance tracking application designed to help users record daily transactions, manage monthly budgets, view summaries, and analyze spending patterns. The application operates fully offline, ensures immediate data saving, and supports Windows, macOS, and Linux systems. This manual explains how to use all features of SmartBudget, including:

- Recording, editing, and deleting income or expense transactions
- Browsing and filtering financial history
- Setting category-based monthly budgets
- Receiving automatic overspending alerts
- Viewing summaries and category charts
- Understanding how data is loaded and saved

5.2 Getting Started

5.2.1 System Requirements

- **Operating Systems:** Windows, macOS, Linux
- **Java Runtime:** Any supported Java runtime compatible with JavaFX
- **Network:** Not required (SmartBudget functions fully offline)

5.2.2 Installation & Setup

5.2.2.1 Prerequisites

Before running SmartBudget, make sure you have:

Required

- Java JDK 21 or newer
- Git

Optional:

- IntelliJ IDEA (recommended IDE)

5.2.2.2 Clone the Repository

- git clone <YOUR_REPOSITORY_URL>

- Then, in the terminal run
  - cd smart-budget-personal-finance-tracker/app

5.2.2.3 Maven Wrapper (Recommended)

This project includes the Maven Wrapper. This means you do NOT need to install Maven manually.

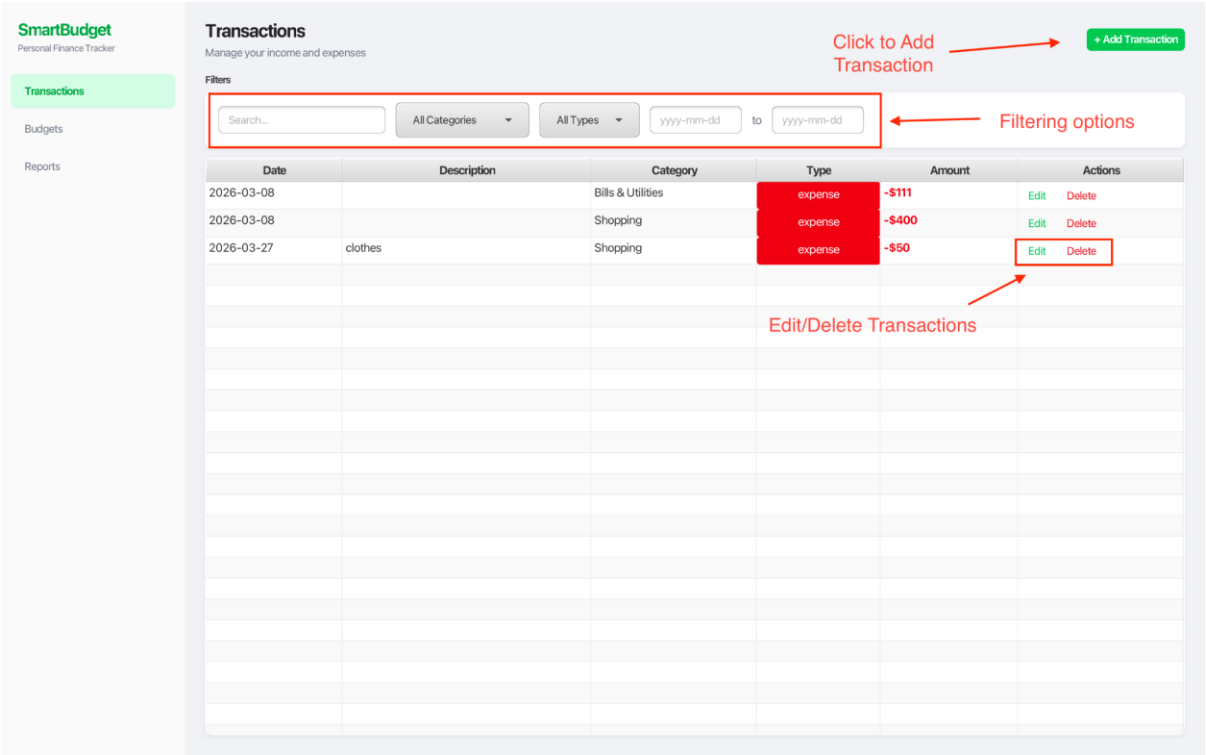
The wrapper automatically downloads and uses the correct Maven version.

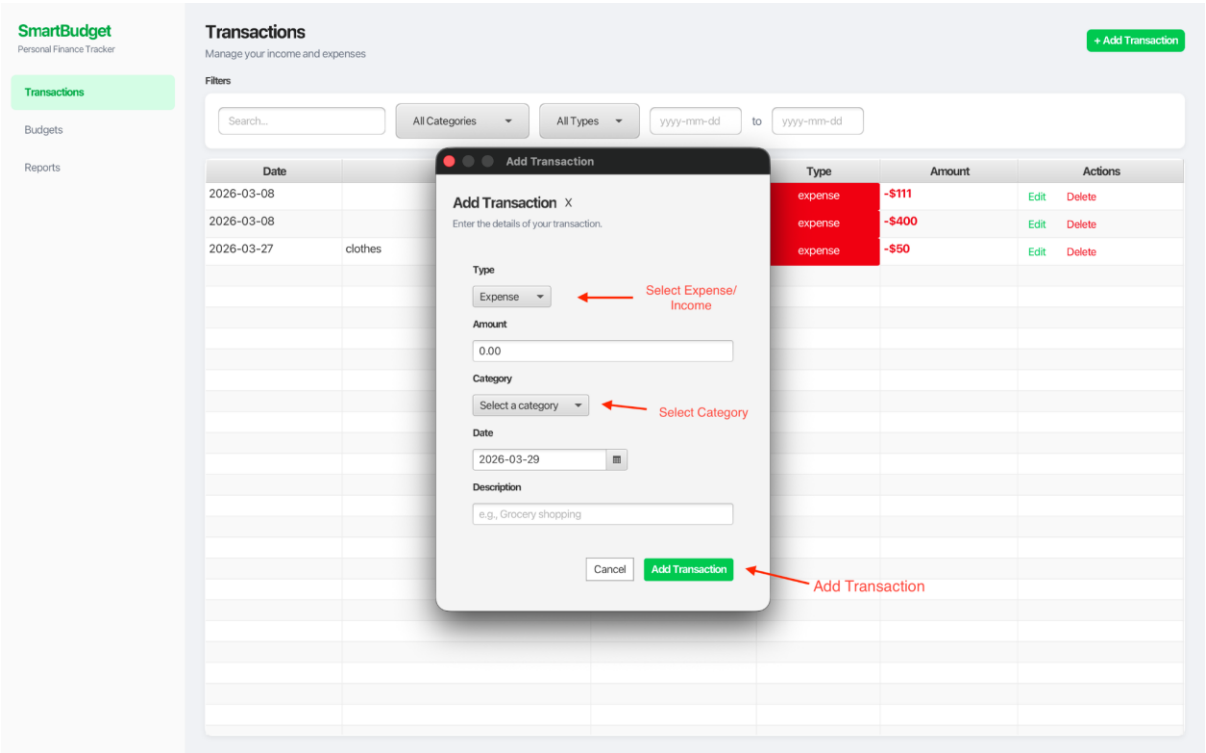
5.2.2.4 Running Inside IntelliJ IDEA

- Open IntelliJ IDEA
- Select Open Project
- Choose the app/ folder
- Allow IntelliJ to import the Maven project
- Ensure Project SDK is set to JDK 21
- For Windows, run in the terminal:
  - ./mvnw.cmd clean test
  - ./mvnw.cmd javafx:run
- For MacOS, run in the terminal:
  - ./mvnw.cmd clean test
  - ./mvnw.cmd javafx:run

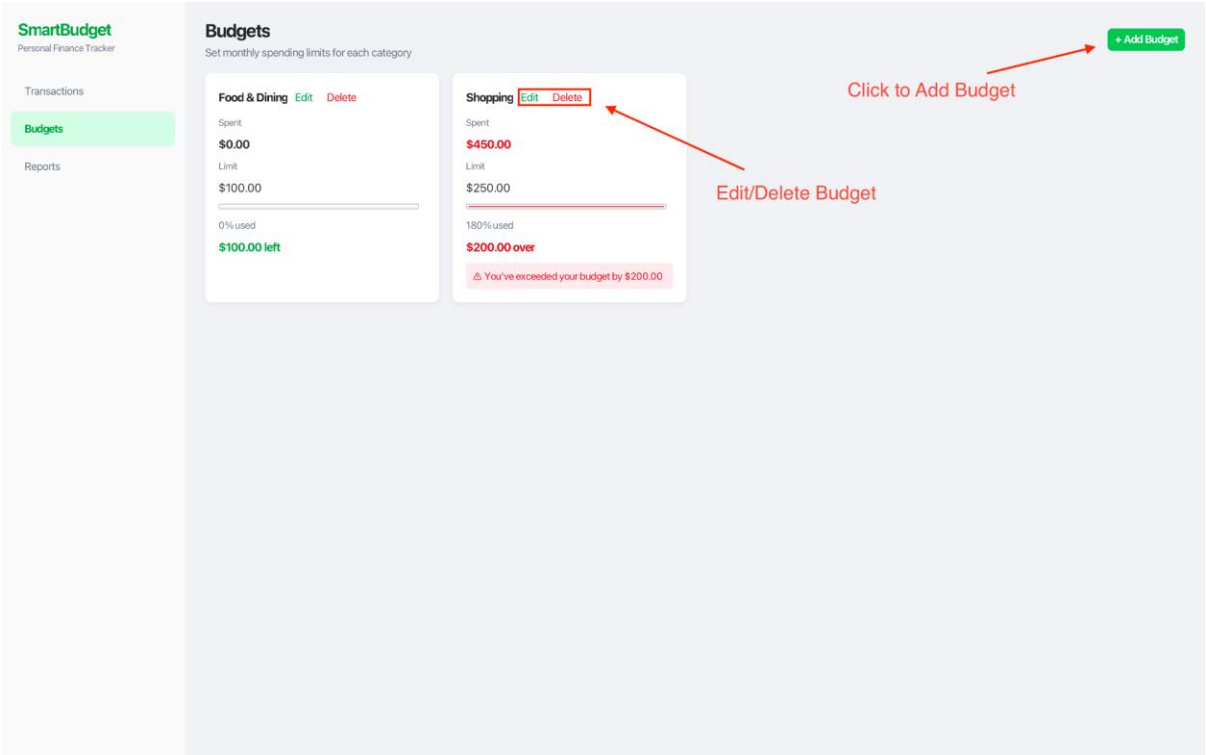
5.3 User Interface Guide

5.3.1 Transactions Page

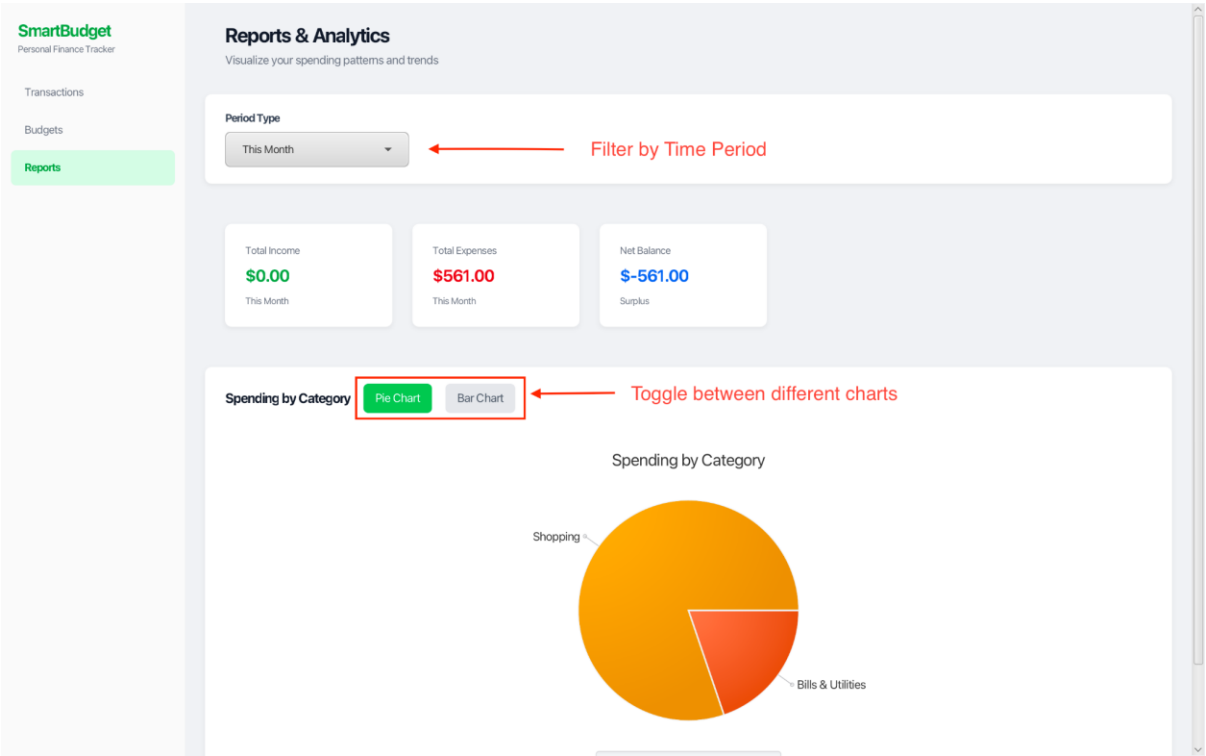




5.3.2 Budgets Page



5.3.3 Reports Page



5.3.4 First Launch

On startup, SmartBudget will automatically:

- Load saved transactions (if any)
- Load monthly budgets (if previously set)
- Display your most recent financial data

If no prior data exists, SmartBudget will inform you that your finance database is empty.

5.4 Navigating SmartBudget

The SmartBudget interface consists of four main sections:

- Dashboard - Summary of Total Income, Expenses, Budgets & Recent Transactions
- Transactions – Record, view, filter, edit, and delete income or expense entries.
- Budgets – Set monthly spending limits per category.
- Reports – View summary totals and category charts.

5.5 Transactions

5.5.1 Adding a Transaction

You can record either an expense or income.

Steps:

1. Open the Transactions tab.
2. Click Add Transaction.
3. Choose Expense or Income.
4. Enter the required details.
5. Click Save.

5.5.2 Editing a Transaction

Steps:

1. Select a transaction from the list.
2. Click Edit.
3. Modify amount, category, date, or note.
4. Save changes.

5.5.3 Deleting a Transaction

Steps:

1. Open the transaction you want to delete.
2. Click Delete.

3. Confirm deletion in the confirmation prompt.

5.5.4 Viewing Transactions

Displays a table of all entries with totals for income, expenses, and balance.

5.5.5 Filtering Transactions

Filter by:

- Category
- Date range
- Combined filters
- Keyword search

Clear Filters resets the list.

5.6 Budgets

5.6.1 Setting a Budget

Steps:

1. Open Budgets tab.
2. Select a category.
3. Enter a monthly limit.
4. Save.
5. Updating a Budget

   Modify the limit and save.

5.7 Overspending Alerts

Alerts appear when spending reaches or exceeds the budget limit for a category.

5.8 Reports

5.8.1 Summary Report

Steps:

1. Open the Reports ab
2. Select reporting period:
   - This Month

- Last Month
- Custom Range

Reports show totals for income, expenses, and balance.

5.8.2 Category Spending Chart

SmartBudget displays at least one visual chart (pie or bar) broken down by category.

Chart Behaviour:

- Automatically updates when transactions change
- Labels and legends clearly identify categories
- Values correspond exactly to underlying data

5.9 Data Persistence

All changes save instantly and load automatically on startup.

5.10 Offline Mode and Privacy

SmartBudget works 100% offline. All data remains local.

5.11 Performance & Responsiveness

Optimized for large datasets with fast loading and filtering.

5.12 Cross-Platform Support

Works uniformly on Windows, macOS, and Linux.

5.13 Maintenance & Updates

Modular architecture, clean code structure, and unit test support.

5.14 Frequently Asked Questions

Question 1: Does SmartBudget work offline?

Yes, completely offline.

Question 2: Is data synced?

Not in the current version. Data is stored locally on each device.

Question 3: How to back up data?

Manually copy the app's storage folder to another location or device.

5.15 Summary

SmartBudget provides a clean, reliable, and intuitive way to:

- Track financial transactions
- Monitor category budgets
- Receive warnings about overspending
- Visualize spending in charts
- Operate securely and offline

This user manual should help you navigate and use all features confidently.

6 Code Repository

The SmartBudget project is hosted on GitHub, and all project deliverables, documents, and source files are stored and updated there throughout each iteration.

6.1 Repository Link:

The full project repository can be accessed at:

https://github.com/Hercules-Hawk/smart-budget-personal-finance-tracker

All project updates have been pushed to this repository.

6.2 Submission Tag

To mark the submission for this deliverable, the current version of the repository has been tagged as: **ITR3.3**

This tag identifies the exact commit corresponding to the Iteration 3.3 submission and allows the instructor and TA to review the specific version of the codebase associated with this deliverable.

7 Conclusion

This final report presents the completed SmartBudget – Personal Finance Tracker project as a fully documented and integrated software system. Across the earlier project iterations, the team moved from project vision and requirements, to design, implementation, testing, and final refinement. In this final submission, all of that work is brought together into one complete document that reflects the finished state of the system and its supporting documentation.

The Software Requirements Specification chapter summarizes the final functional and non-functional requirements of SmartBudget and ensures that the implemented system remains aligned with the original project goals. The Software Design Document chapter then presents the final design of the system, including the architectural structure and the diagrams that describe how the main workflows and components are organized. Together, these chapters show that the project was developed in a structured way, with clear links between requirements, design, and implementation.

The testing chapter demonstrates that the final system was validated through a combination of test planning, functional testing, and integration testing. Earlier testing work established traceability between requirements and test cases, while later iterations expanded this into real execution results for transactions, budgets, alerts, reports, persistence, and full end-to-end workflows. As a result, the report shows that SmartBudget is not only implemented, but also tested in a way that supports confidence in its correctness, reliability, and overall readiness for submission.

The user manual chapter completes the report from the end-user perspective by explaining how to install, navigate, and use the software. This makes the final submission more than a technical report, because it also provides practical guidance for anyone who needs to operate the application and understand its main features.

Overall, SmartBudget has progressed from an initial concept into a complete desktop application that supports the project's core goals: recording and managing transactions, setting monthly budgets, receiving overspending alerts, generating summary reports, and preserving user data across sessions. The final report therefore serves as a complete record of the project's development journey and demonstrates that the system, its documentation, and its testing have been brought together into a final professional submission